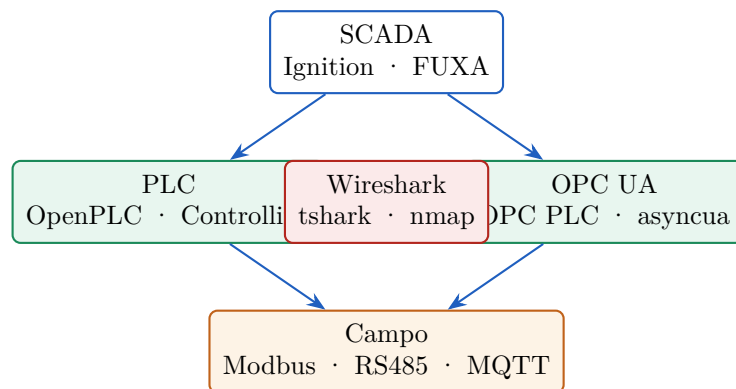


UNIVERSIDAD DE CUENCA
Maestría en Ciencias de la Ingeniería
Informática Industrial



Laboratorio de redes industriales en Docker

Modbus · OPC UA · MQTT · SCADA · análisis de tráfico

Guía de instalación, marco teórico y prácticas

Pablo Barbecho
pablo.barbecho@ucuenca.edu.ec

7 de septiembre de 2026

Índice general

1. Introducción	6
1.1. Motivación	6
1.2. Objetivos	6
1.3. Alcance y organización del documento	6
1.4. Convenciones	7
2. Marco teórico	8
2.1. Redes industriales y la pirámide de automatización	8
2.2. Contenedores: Docker y Docker Compose	9
2.2.1. Conceptos	9
2.2.2. Redes	9
2.2.3. Volúmenes: bind mounts frente a volúmenes con nombre	10
2.2.4. Arquitecturas de CPU e imágenes multi-arquitectura	10
2.3. Modbus	11
2.3.1. Origen y modelo	11
2.3.2. Modelo de datos	11
2.3.3. Trama Modbus TCP	11
2.3.4. Seguridad	12
2.4. OPC UA	12
2.4.1. Qué resuelve	12
2.4.2. Modelo de información	12
2.4.3. Pila de comunicaciones y seguridad	12
2.4.4. Descubrimiento y endpoints	13
2.5. MQTT	13
2.6. Herramientas de control (nivel 1)	14
2.6.1. OpenPLC	14
2.6.2. OPC PLC (Microsoft)	14
2.6.3. Servidor OPC UA en Python (asyncua)	14
2.7. Herramientas de supervisión (nivel 2)	14
2.7.1. Ignition	14
2.7.2. FUXA	15
2.7.3. Node-RED	15
2.8. Herramientas de análisis y seguridad	15
2.8.1. Wireshark, tshark y tcpdump	15
2.8.2. netshoot	16
2.8.3. nmap y scripts NSE	16
2.8.4. Conpot	16
2.8.5. UaExpert	16
2.8.6. Monitorización pasiva y port mirroring	16
2.9. Hardware físico	17
2.9.1. Controllino	17

2.9.2. ESP32 PLC 14 (Industrial Shields)	17
2.9.3. RS485 y Modbus RTU	17
2.10. Ethernet industrial en tiempo real	18
3. Licencias y costes	19
3.1. Ignition	20
3.1.1. Modo trial (edición Standard)	20
3.1.2. Maker Edition	20
3.1.3. Licencias educativas y comerciales	20
3.2. Docker Desktop	21
3.3. UaExpert y otras herramientas de escritorio	21
4. Instalación y puesta en marcha	22
4.1. Requisitos	22
4.2. Elección de plataforma	22
4.3. Instalación de Docker	23
4.3.1. macOS (Docker Desktop)	23
4.3.2. Linux (Ubuntu/Debian)	23
4.3.3. Windows	23
4.4. Estructura del proyecto	24
4.5. Arranque	25
4.5.1. Comprobaciones rápidas	25
4.6. Primer acceso a Ignition	26
4.7. Problemas encontrados en la puesta en marcha y su solución	26
4.7.1. Imágenes solo amd64 en Apple Silicon	26
4.7.2. Conpot: <code>exec: "conpot": executable file not found in \$PATH</code>	27
4.7.3. <code>modbus-sim: KeyError: 'logging'</code>	27
4.7.4. Ignition: <code>cp: cannot stat './data/gateway.xml_clean'</code>	27
4.8. Operación diaria	27
5. Prácticas guiadas	29
5.1. Práctica 1 — Modbus TCP	30
5.1.1. Esclavo simulado: <code>modbus-sim</code>	30
5.1.2. Cliente Modbus desde un contenedor	30
5.1.3. PLC virtual: OpenPLC	31
5.2. Práctica 2 — OPC UA	31
5.2.1. Servidor simulado: OPC PLC	31
5.2.2. Servidor propio en Python	31
5.2.3. Cliente UaExpert	31
5.3. Práctica 3 — SCADA	32
5.3.1. Ignition	32
5.3.2. FUXA	32
5.3.3. Node-RED	33
5.4. Práctica 4 — Captura y análisis de tráfico	33
5.4.1. Capturar dentro de la red Docker con <code>netshoot</code>	33
5.4.2. Capturar desde el host (solo Linux)	33
5.4.3. Filtros útiles en Wireshark	34
5.4.4. Reconocimiento con <code>nmap</code>	34
5.5. Práctica 5 — Honeypot ICS con Conpot	34

6. Práctica 6 — Integración de los PLC físicos	36
6.1. Equipos	36
6.2. Topología física y direccionamiento	37
6.3. Cómo llegan los contenedores a los PLC	37
6.4. Firmware: opción rápida con OpenPLC Editor	38
6.5. Firmware: opción con Arduino IDE (control total)	38
6.6. Modbus RTU sobre RS485 entre los dos PLC	40
6.7. MQTT desde el ESP32	40
7. Ejercicios propuestos	41
7.1. Con el laboratorio en Docker	41
7.2. Con los PLC físicos	42
8. Problemas frecuentes	43
A. Archivos de configuración	45
A.1. docker-compose.yml (versión macOS / Docker Desktop)	45
A.2. docker-compose.windows.yml (Windows / WSL 2)	47
A.3. docker-compose.yml (versión original Linux)	50
A.4. modbus/server.json	51
A.5. opcua/server.py y opcua/Dockerfile	52
A.6. mosquitto/mosquitto.conf	52
B. Referencia rápida de comandos	54

Índice de figuras

1.1.	Orden de montaje del laboratorio: cada paso añade una capa sobre la anterior. . .	7
2.1.	Niveles de la pirámide de automatización y herramienta del laboratorio que ocupa cada uno.	8
2.2.	Red bridge de Docker en Linux frente a Docker Desktop (macOS/Windows). En Linux el bridge es una interfaz real del host; en macOS vive dentro de una máquina virtual y solo se accede por los puertos publicados.	10
2.3.	Comportamiento de Docker al montar un directorio vacío sobre una ruta que la imagen ya rellenaba.	10
2.4.	Estructura de una petición Modbus TCP: cabecera MBAP y PDU.	11
2.5.	Pila OPC UA sobre transporte binario. El contraste entre una sesión <code>None/None</code> y una <code>SignAndEncrypt</code> es el ejercicio central de OPC UA en el laboratorio.	13
2.6.	Modelo publicación-suscripción de MQTT en el laboratorio.	13
2.7.	Arquitectura de Ignition: un Gateway concentra drivers, OPC UA, tags y módulos de visualización; el Designer y las sesiones Perspective son clientes del Gateway. . .	15
2.8.	Puntos de captura del laboratorio y qué tráfico se observa en cada uno.	17
3.1.	Elección de la vía de licenciamiento de Ignition para el laboratorio.	20
4.1.	Secuencia de arranque y verificación del laboratorio.	25
5.1.	Topología completa del laboratorio: contenedores Docker en el host y segmento OT físico con Controllino y ESP32 PLC 14.	29
5.2.	Flujo de datos y protocolos entre los componentes del laboratorio.	30
6.1.	Conexiones físicas: alimentación, Ethernet, bus RS485 y USB de programación. . .	37

Índice de tablas

2.1. Diferencias entre redes IT y OT relevantes para el laboratorio.	9
2.2. Tablas de datos Modbus y códigos de función asociados.	11
2.3. Opciones para estudiar Ethernet industrial de tiempo real fuera de Docker.	18
3.1. Licencia y coste de cada componente del laboratorio.	19
4.1. Requisitos de software y hardware.	22
4.2. Servicios del laboratorio, direcciones internas, puertos publicados y credenciales.	25
5.1. Registros iniciales del esclavo simulado.	30
5.2. Filtros de visualización de Wireshark para el laboratorio.	34
6.1. Características de los dos PLC físicos y su papel en el laboratorio.	36
8.1. Problemas frecuentes y solución.	43

Capítulo 1

Introducción

1.1. Motivación

Las redes industriales son el sistema nervioso de una planta: por ellas viajan las lecturas de sensores, las órdenes a actuadores, los datos que alimentan al SCADA y, cada vez más, la información que sube a la nube. Estudiarlas exige ver los protocolos “en el cable”, no solo en el diagrama; y montar una planta real para ello es caro, lento y arriesgado. Este laboratorio resuelve ese problema ejecutando en un solo PC, mediante contenedores Docker, todos los componentes lógicos de una arquitectura de automatización moderna, e integrándolos después con dos controladores físicos de bajo coste (Controllino y ESP32 PLC 14 de Industrial Shields) para que la experiencia no sea puramente simulada.

El énfasis es de *redes*: cada práctica termina con una captura de tráfico y con una pregunta sobre qué se ve, qué no se ve y por qué. Los protocolos se estudian tal como aparecen en Wireshark, y la seguridad (segmentación, reconocimiento, honeypots) se trata como parte inseparable del diseño de la red.

1.2. Objetivos

Al completar la guía el estudiante será capaz de:

- Desplegar y administrar con Docker Compose una red de laboratorio con dispositivos de campo simulados, un PLC virtual, servidores OPC UA, un broker MQTT, dos plataformas SCADA y un honeypot industrial.
- Explicar la estructura de los protocolos Modbus TCP/RTU, OPC UA y MQTT e identificar sus mensajes en una captura de red.
- Configurar Ignition, FUXA y Node-RED como clientes de esos protocolos y construir una pantalla HMI mínima.
- Capturar tráfico dentro de la red Docker, desde el host y desde un puerto espejo de un switch, y analizarlo con Wireshark, tshark y nmap.
- Integrar controladores físicos (Controllino, ESP32) en la red del laboratorio por Ethernet, WiFi y RS485, y comparar su comportamiento con los dispositivos simulados.
- Razonar sobre segmentación, detección y licenciamiento de las herramientas empleadas.

1.3. Alcance y organización del documento

La guía se organiza en tres bloques. El **marco teórico** (capítulo 2) explica cada tecnología y herramienta con el detalle necesario para entender lo que ocurre en las prácticas. El bloque de **puesta en marcha** (capítulos 3 y 4) cubre licencias, requisitos, instalación en macOS y Linux, y los problemas concretos que aparecieron al desplegar el laboratorio, con su solución. El bloque **práctico** (capítulos 5 a 7) recorre paso a paso Modbus, OPC UA, SCADA, captura de tráfico,

honeypot y hardware real, y propone catorce ejercicios graduados. El capítulo 8 recopila fallos frecuentes y los anexos contienen todos los archivos de configuración completos.

La figura 1.1 muestra el orden de montaje recomendado: cada paso añade una capa sobre la anterior, de modo que el laboratorio es utilizable desde el paso 2.

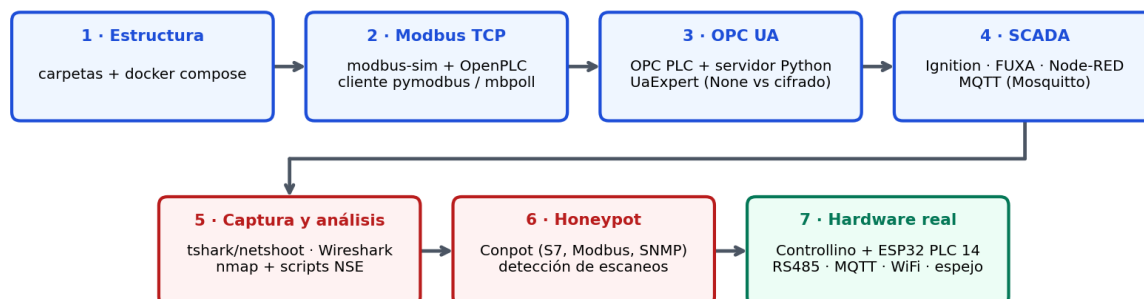


Figura 1.1: Orden de montaje del laboratorio: cada paso añade una capa sobre la anterior.

1.4. Convenciones

- Los comandos se muestran en bloques de código; el símbolo \$ no se escribe. Todos se ejecutan desde la carpeta `ot-lab` salvo indicación contraria.
- Las rutas de menú se escriben como **Config** → **OPC UA** → **Connections**.
- Las direcciones IP de la red Docker son `172.28.0.x`; las del segmento físico OT, `192.168.10.x`. Desde el host (macOS o Windows) los servicios se alcanzan siempre por `localhost:<puerto publicado>`.
- Las cajas de color destacan notas, avisos, errores críticos y resultados esperados.

Archivos de referencia

Todo el laboratorio vive en la carpeta `ot-lab`. Los archivos íntegros (`docker-compose.yml`, `modbus/server.json`, `opcua/server.py`, `mosquitto/mosquitto.conf`) están reproducidos en el anexo A y en la subcarpeta `docs/latex/src/` de esta guía.

Capítulo 2

Marco teórico

Este capítulo describe las tecnologías y herramientas que componen el laboratorio. Se agrupan siguiendo la pirámide de automatización: primero el modelo de referencia, luego la plataforma de contenedores que lo aloja, después los tres protocolos (Modbus, OPC UA, MQTT), las herramientas de control y supervisión, las de análisis y seguridad y, por último, el hardware físico.

2.1. Redes industriales y la pirámide de automatización

El modelo de referencia más usado para describir una planta es la *pirámide de automatización* (ISA-95 / modelo de Purdue), que organiza los sistemas en niveles según su función y su horizonte temporal (figura 2.1). El **nivel 0** son sensores y actuadores; el **nivel 1**, los controladores (PLC, RTU) que ejecutan lazos en milisegundos; el **nivel 2**, la supervisión (SCADA/HMI, historiadores) que sondea a los controladores en el orden de segundos; el **nivel 3**, la gestión de operaciones (MES); y el **nivel 4**, los sistemas corporativos (ERP). Entre los niveles 3 y 4 la práctica moderna inserta una *DMZ industrial* (nivel 3.5) que rompe la conectividad directa entre la red de planta (OT) y la red de empresa (IT).

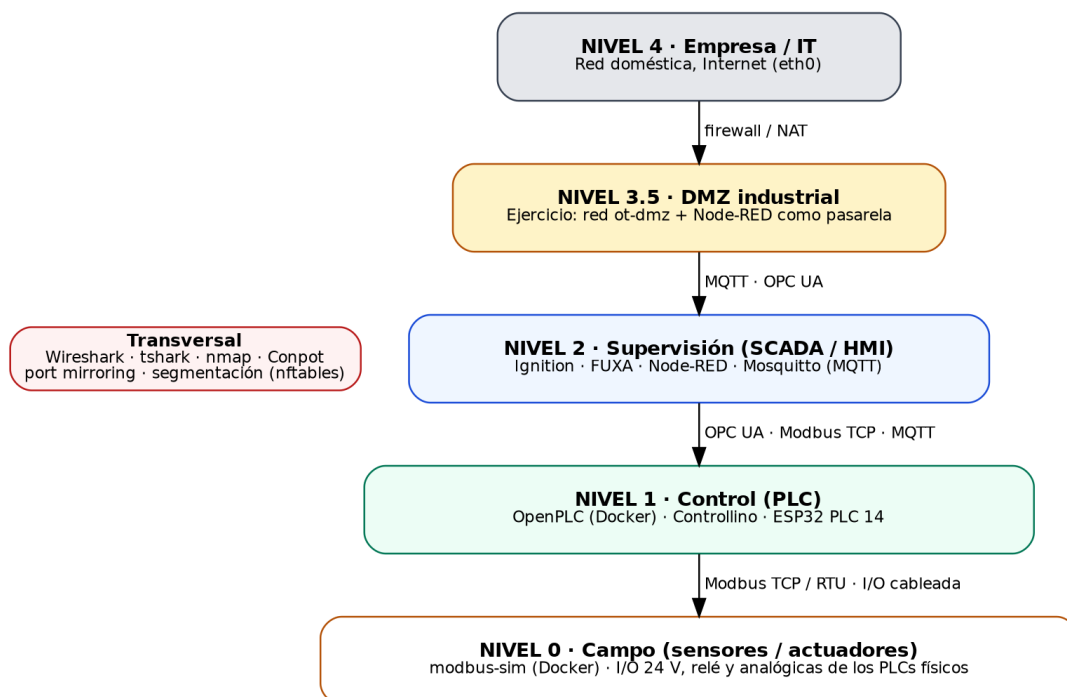


Figura 2.1: Niveles de la pirámide de automatización y herramienta del laboratorio que ocupa cada uno.

Las redes OT difieren de las IT en varios aspectos que condicionan todo el laboratorio (tabla 2.1). La prioridad es la disponibilidad y el determinismo, no la confidencialidad; los equipos viven décadas y rara vez se parchean; los protocolos nacieron sin autenticación porque asumían un cable aislado. Por eso el análisis de tráfico es tan revelador en OT: casi todo viaja en claro y es interpretable.

Tabla 2.1: Diferencias entre redes IT y OT relevantes para el laboratorio.

Aspecto	Red IT	Red OT
Prioridad	Confidencialidad, integridad	Disponibilidad, seguridad física, determinismo
Patrón de tráfico	Ráfagas, cliente-servidor variado	Sondeo periódico (<i>polling</i>) muy regular
Ciclo de vida	3-5 años	15-25 años
Autenticación	Habitual (TLS, Kerberos)	Ausente en Modbus, S7, EtherNet/IP clásico; opcional en OPC UA y MQTT
Latencia tolerable	Cientos de ms	De 1 ms (control de movimiento) a 1 s (supervisión)
Consecuencia de un fallo	Pérdida de datos	Parada de planta, daño físico

2.2. Contenedores: Docker y Docker Compose

2.2.1. Conceptos

Un **contenedor** es un proceso aislado del resto del sistema mediante *namespaces* (PID, red, sistema de archivos) y *cgroups* (límites de CPU y memoria) del kernel Linux. A diferencia de una máquina virtual no lleva su propio kernel, por lo que arranca en milisegundos y consume solo la memoria de la aplicación. Una **imagen** es la plantilla inmutable, construida por capas a partir de un **Dockerfile**, de la que se instancian contenedores. Un **registro** (Docker Hub, mcr.microsoft.com) almacena y distribuye imágenes.

Docker Compose describe en un archivo YAML un conjunto de servicios, redes y volúmenes y los gestiona como una unidad (`docker compose up/down/ps/logs`). En este laboratorio, `docker-compose.yml` define nueve servicios sobre una red común.

2.2.2. Redes

Docker crea por defecto redes de tipo *bridge*: un conmutador virtual (en Linux, una interfaz `br-<id>`) al que se conecta la interfaz `eth0` de cada contenedor. Los contenedores de la misma red se ven entre sí por IP y por nombre de servicio (Docker incluye un DNS interno). Para que el host o el exterior alcancen un servicio se *publica* un puerto ("`8088:8088`"), lo que instala una regla NAT del puerto del host al del contenedor. En el laboratorio asignamos IPs fijas (`ipv4_address`) en la subred `172.28.0.0/24` para que las capturas y los escaneos sean legibles y repetibles.

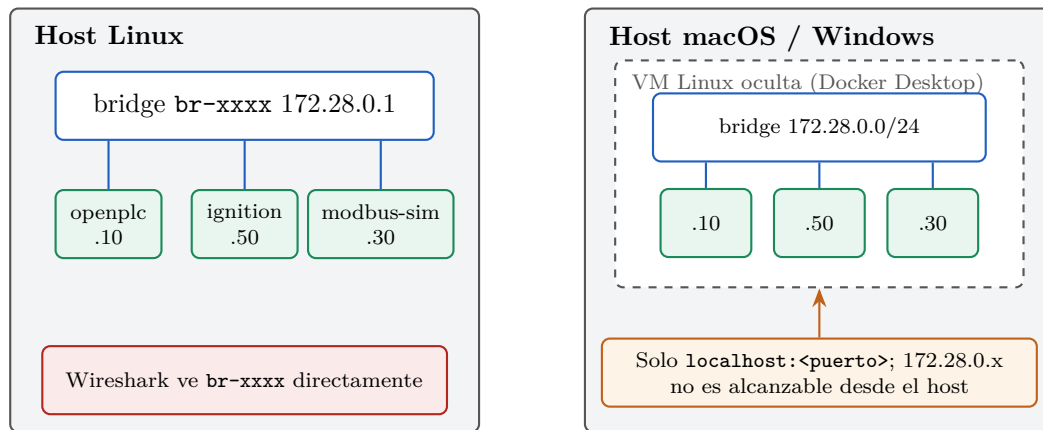


Figura 2.2: Red bridge de Docker en Linux frente a Docker Desktop (macOS/Windows). En Linux el bridge es una interfaz real del host; en macOS vive dentro de una máquina virtual y solo se accede por los puertos publicados.

Implicación para macOS y Windows

En Docker Desktop la red 172.28.0.x no es alcanzable desde el host y no existe una interfaz **br-xxxx** que Wireshark pueda abrir. Las capturas se hacen *dentro* de la red Docker con un contenedor auxiliar (**netshoot**, sección 2.8.2) o en el segmento físico. Entre contenedores las IPs fijas funcionan con normalidad.

2.2.3. Volúmenes: bind mounts frente a volúmenes con nombre

El sistema de archivos de un contenedor es efímero. Para persistir datos hay dos mecanismos que se comportan de forma distinta al arrancar sobre un directorio vacío, y esa diferencia fue la causa de uno de los fallos de la puesta en marcha (sección 4.7):

- **Bind mount** (./carpeta:/ruta): monta una carpeta del host sobre la ruta del contenedor. Lo que hubiera en la imagen en esa ruta queda *oculto*; si la carpeta del host está vacía, el contenedor ve un directorio vacío.
- **Volumen con nombre** (nombre:/ruta): gestionado por Docker. La primera vez que se monta vacío, Docker *copia* en él el contenido que la imagen tenía en esa ruta. Es el mecanismo que esperan imágenes como Ignition, que traen un directorio **data/** preconfigurado.

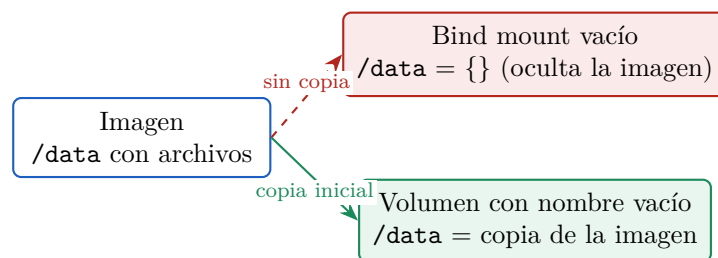


Figura 2.3: Comportamiento de Docker al montar un directorio vacío sobre una ruta que la imagen ya rellenaba.

2.2.4. Arquitecturas de CPU e imágenes multi-arquitectura

Una imagen contiene binarios compilados para una arquitectura. Los Mac con Apple Silicon (y la Raspberry Pi) son **arm64**; la mayoría de los PC, **amd64**. Muchas imágenes publican un *manifiesto multi-arquitectura* y Docker escoge la variante adecuada. Cuando solo existe **amd64** (caso de **tuttas/openplc_v3** y **honeynet/conpot**), en un Mac ARM hay que indicarlo con **platform:**

linux/amd64: Docker Desktop la ejecuta mediante emulación (QEMU o Rosetta 2), más lenta pero funcional. Sin esa línea el error es `exec format error`.

2.3. Modbus

2.3.1. Origen y modelo

Modbus (Modicon, 1979) es el protocolo industrial más extendido por su simplicidad y por ser de especificación abierta. Sigue un modelo **maestro-esclavo** (en la terminología actual, cliente-servidor): el maestro envía una petición y un único esclavo responde; los esclavos nunca hablan espontáneamente. Existen dos variantes principales:

- **Modbus RTU**: sobre línea serie (habitualmente RS485 half-duplex, 9600–115200 bps, 8N1). Un bus multipunto con hasta 247 esclavos identificados por su *unit ID*. La trama se delimita por silencios de al menos 3,5 caracteres y termina con un CRC-16.
- **Modbus TCP**: la misma PDU encapsulada en TCP (puerto 502) con una cabecera adicional, la **MBAP** (*Modbus Application Protocol header*). Sin CRC (lo aporta TCP) y con un *transaction ID* que permite peticiones concurrentes.

2.3.2. Modelo de datos

Un esclavo expone cuatro tablas (tabla 2.2). Las direcciones de protocolo van de 0 a 65535 en cada tabla. Históricamente se usaba una notación con prefijo (*coils* 0xxxx, entradas discretas 1xxxx, registros de entrada 3xxxx, *holding* 4xxxx, con base 1), de modo que “40001” significa *holding register* de offset 0. Esta doble notación es fuente permanente de confusión y apareció en la puesta en marcha del simulador (sección 4.7): el archivo de configuración esperaba offsets de protocolo, no direcciones 4xxxx.

Tabla 2.2: Tablas de datos Modbus y códigos de función asociados.

Tabla	Tamaño	Acceso	Notación 1-based	Funciones (FC)
Coils (salidas discretas)	1 bit	R/W	00001–09999	01 leer · 05 escribir uno · 15 escribir varios
Discrete inputs	1 bit	R	10001–19999	02 leer
Input registers	16 bit	R	30001–39999	04 leer
Holding registers	16 bit	R/W	40001–49999	03 leer · 06 escribir uno · 16 escribir varios · 23 leer/escribir

2.3.3. Trama Modbus TCP

La figura 2.4 muestra la estructura de una petición Modbus TCP. La MBAP ocupa 7 bytes: *transaction ID* (2), *protocol ID* (2, siempre 0), *length* (2, bytes que siguen) y *unit ID* (1, dirección del esclavo, relevante en pasarelas TCP→RTU). Le sigue la PDU: código de función (1) y datos. En Wireshark el disector `mbtcp` muestra la MBAP y `modbus` la PDU, con los filtros `modbus.func_code` y `modbus.exception_code`.

Ejemplo: 00 01 | 00 00 | 00 06 | 01 | 03 | 00 00 | 00 03 ⇒ leer 3 registros desde el offset 0 del esclavo 1

Transaction ID 2 bytes	Protocol ID 2 bytes = 0	Length 2 bytes	Unit ID 1 byte	Function 1 byte	Start addr. 2 bytes	Quantity 2 bytes
MBAP (7 bytes)				PDU (FC 03: leer holding registers)		

Figura 2.4: Estructura de una petición Modbus TCP: cabecera MBAP y PDU.

Si la petición no puede atenderse, el esclavo responde con el código de función más 0x80 y un *exception code* (01 función ilegal, 02 dirección ilegal, 03 valor ilegal, 04 fallo del dispositivo). Provocar una excepción leyendo un registro inexistente es uno de los primeros ejercicios de la guía.

2.3.4. Seguridad

Modbus carece de autenticación, autorización y cifrado: cualquier equipo con acceso a la red puede leer y escribir cualquier registro. Existe una variante Modbus/TCP Security (puerto 802, sobre TLS con certificados) con adopción muy escasa. En la práctica la protección viene de la red: segmentación, listas de acceso que solo permitan al SCADA hablar con los PLC y monitorización pasiva que alerte de escrituras (FC 05/06/15/16) desde orígenes no autorizados.

2.4. OPC UA

2.4.1. Qué resuelve

OPC UA (IEC 62541, 2008) sustituye al OPC clásico basado en DCOM de Windows por un estándar independiente de plataforma para el intercambio de datos y metadatos en automatización. Frente a Modbus, aporta tres cosas: un **modelo de información** rico (los datos tienen nombre, tipo, unidades, jerarquía, no son registros anónimos), **servicios** de alto nivel (lectura, escritura, suscripción, métodos, alarmas, histórico) y **seguridad** integrada (certificados X.509, firma y cifrado, autenticación de usuario).

2.4.2. Modelo de información

Un servidor expone un *espacio de direcciones* formado por nodos (objetos, variables, métodos, tipos) conectados por referencias. Cada nodo tiene un *NodeId* (`ns=2;i=1001` o `ns=2;s=Tanque1.Nivel`) dentro de un *namespace*. Los clientes navegan desde `Objects` igual que por un árbol de carpetas; UaExpert e Ignition lo hacen gráficamente. El servidor Python del laboratorio crea el objeto `Tanque1` con las variables `Nivel`, `Temperatura` y `Valvula` en el namespace `http://lab.local`.

2.4.3. Pila de comunicaciones y seguridad

La figura 2.5 muestra la pila del transporte binario `opc.tcp` (puerto habitual 4840). Sobre TCP se establece un **SecureChannel** negociando una *política de seguridad* (`None`, `Basic256Sha256`, `Aes256_Sha256_RsaPss`) y un *modo* (`None`, `Sign`, `SignAndEncrypt`) mediante intercambio de certificados de aplicación. Dentro del canal se abre una **Session** donde el usuario se autentica (anónimo, usuario/contraseña o certificado). Solo entonces circulan los servicios (`Browse`, `Read`, `Write`, `CreateSubscription`, `Publish`).

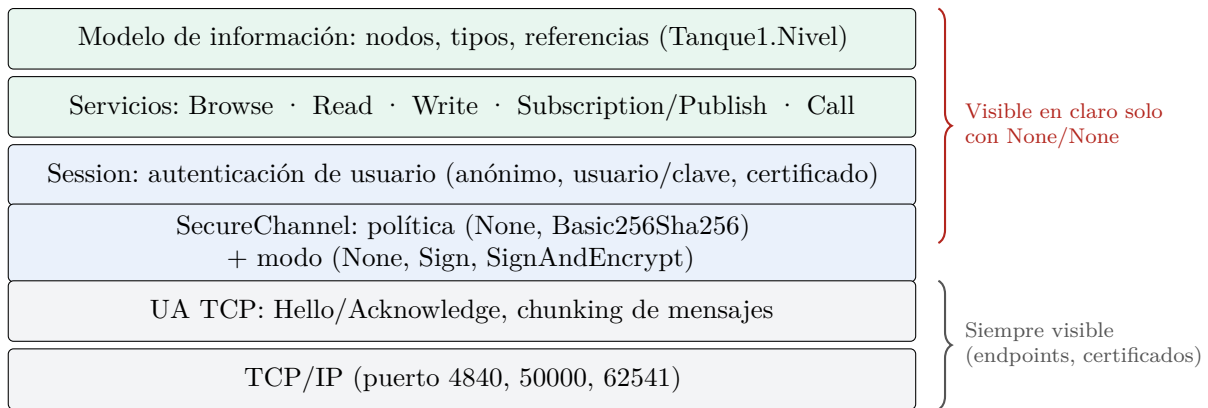


Figura 2.5: Pila OPC UA sobre transporte binario. El contraste entre una sesión *None/None* y una *SignAndEncrypt* es el ejercicio central de OPC UA en el laboratorio.

2.4.4. Descubrimiento y endpoints

Un cliente primero pide al servidor sus *endpoints* (*GetEndpoints*): la lista de combinaciones URL, política y modo que acepta. Esa respuesta viaja siempre en claro, por lo que un escáner puede saber qué políticas ofrece un servidor sin autenticarse. El servidor OPC PLC de Microsoft acepta cualquier certificado de cliente si se lanza con *-autoaccept*; en producción el administrador debe mover manualmente cada certificado de cliente a la lista de confianza.

2.5. MQTT

MQTT (1999, hoy estándar OASIS/ISO 20922) es un protocolo ligero de **publicación-suscripción** pensado para enlaces poco fiables. A diferencia de Modbus y OPC UA, no hay conexión directa entre productor y consumidor: ambos se conectan a un **broker** (Mosquitto en el laboratorio, puerto 1883; 8883 con TLS). Los publicadores envían mensajes a un *topic* jerárquico (*ot/esp32/nivel*) y el broker los reenvía a quien esté suscrito, con comodines *+* (un nivel) y *#* (todo lo que sigue). Tres niveles de QoS regulan la entrega (0: a lo sumo una vez; 1: al menos una vez; 2: exactamente una vez), y los mensajes *retained* permiten que un nuevo suscriptor reciba el último valor al conectarse.

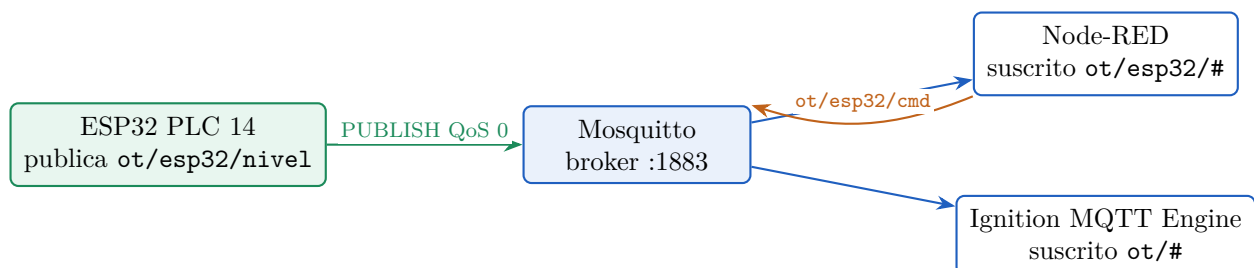


Figura 2.6: Modelo publicación-suscripción de MQTT en el laboratorio.

En planta MQTT se usa como capa IIoT (Sparkplug B define una carga útil y una semántica de estado para ello). Sin TLS, todo el payload es legible en Wireshark con el filtro *mqtt*; el ejercicio del capítulo 7 compara 1883 con 8883.

2.6. Herramientas de control (nivel 1)

2.6.1. OpenPLC

OpenPLC es un runtime de PLC de código abierto conforme a IEC 61131-3 (lenguajes LD, FBD, ST, IL, SFC). Se compone del **OpenPLC Editor** (herramienta de escritorio basada en Beremiz para escribir y compilar programas) y del **OpenPLC Runtime**, que ejecuta el programa compilado y expone una interfaz web (puerto 8080) para cargarlo, arrancarlo y configurar comunicaciones. El runtime actúa como **esclavo Modbus TCP** (puerto 502) publicando sus variables en las direcciones IEC (%IX0.0 entradas digitales, %QX0.0 salidas, %IW100 registros de entrada, %QW100 *holding*), y también como **maestro** (menú **Slave Devices**) que sondea otros esclavos Modbus y mapea sus registros en las mismas direcciones. Esa doble función es lo que permite que el PLC virtual en Docker lea el simulador y controle los PLC físicos. El Editor, además, compila programas para placas Arduino, Controllino y ESP32 dejándolas como esclavos Modbus TCP/RTU.

2.6.2. OPC PLC (Microsoft)

`mcr.microsoft.com/iotedge/opc-plc` es un servidor OPC UA de simulación creado para probar Azure IoT. Genera nodos con distintos comportamientos configurables por línea de comandos: `-sn` nodos lentos, `-fn` rápidos, `-gn` con “fallos” (picos, valores fuera de rango), `-sph` publica su propio certificado. Las opciones `-autoaccept`, `-ut` (usuarios anónimos) y `-dca` (no comprobar el dominio del certificado) relajan la seguridad para laboratorio y deben retirarse para practicar certificados. Escucha en el puerto 50000.

2.6.3. Servidor OPC UA en Python (asyncua)

`asyncua` (*opcua-asyncio*) es la biblioteca Python de referencia para clientes y servidores OPC UA. El servidor del laboratorio (`opcua/server.py`, anexo A.5) tiene 25 líneas: registra un namespace, crea un objeto con tres variables y actualiza dos de ellas cada segundo con funciones senoidales. Su valor didáctico es doble: muestra cómo se construye un espacio de direcciones y proporciona un servidor cuyo código se puede modificar (añadir métodos, exigir seguridad, cambiar tipos) para observar el efecto en la red.

2.7. Herramientas de supervisión (nivel 2)

2.7.1. Ignition

Ignition (Inductive Automation) es una plataforma SCADA/IIoT comercial basada en Java con arquitectura cliente-servidor centrada en el **Gateway** (figura 2.7). El Gateway es un único proceso que integra los *drivers* de dispositivos (Modbus TCP, Siemens S7, Allen-Bradley Logix, Omron, DNP3, BACnet), un cliente y un servidor OPC UA, el sistema de *tags*, el historiador (sobre una base de datos SQL), alarmas, scripting Python (Jython) y los módulos de visualización. Todo se administra por web (puerto 8088). El **Designer** es la herramienta de desarrollo, que se lanza desde el Gateway mediante el *Designer Launcher*. **Perspective** es el módulo de HMI web (HTML5, responsive) y **Vision** el módulo clásico de cliente Java de escritorio.

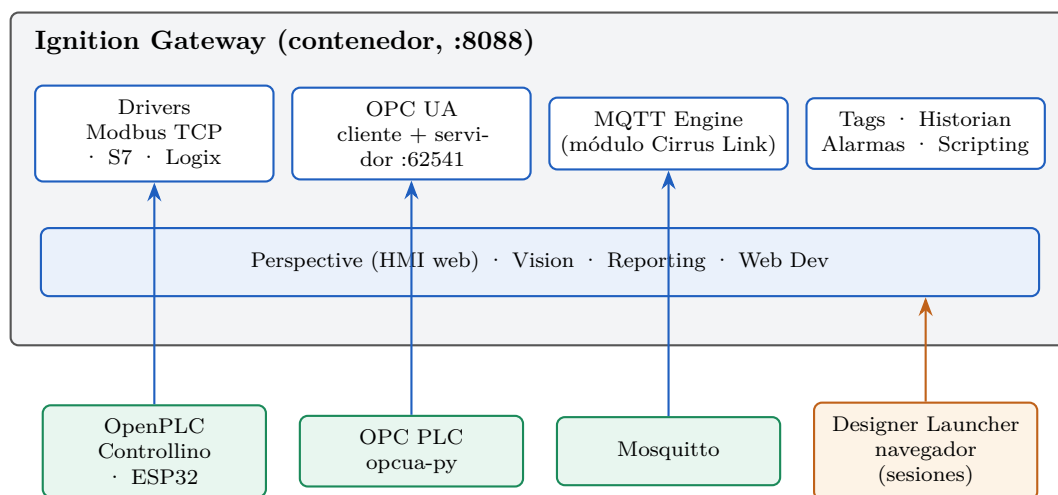


Figura 2.7: Arquitectura de Ignition: un Gateway concentra drivers, OPC UA, tags y módulos de visualización; el Designer y las sesiones Perspective son clientes del Gateway.

Ignition es relevante en un curso de redes porque su comportamiento es el de un SCADA de planta real: abre varias conexiones TCP por dispositivo, sondea a intervalos configurables (*scan class / tag group*), reintenta ante fallos y genera la “firma” de tráfico periódico que un analista debe reconocer. Su modelo de licencias se trata en el capítulo 3.

2.7.2. FUXA

FUXA es un SCADA/HMI web de código abierto (Node.js + Angular) mucho más ligero que Ignition. Ofrece conexiones Modbus TCP/RTU, OPC UA, S7, MQTT, BACnet y WebAPI, un editor gráfico de sinópticos SVG en el navegador (puerto 1881), alarmas y un histórico sencillo. Sirve como segundo cliente para contrastar cómo dos SCADA distintos sondean el mismo esclavo (número de conexiones, intervalo, agrupación de registros) y como alternativa cuando Ignition no está disponible.

2.7.3. Node-RED

Node-RED es un entorno de programación por flujos (nodos conectados en un editor web, puerto 1880) construido sobre Node.js. Con los paquetes `node-red-contrib-modbus` y `node-red-contrib-opcua` se convierte en una pasarela de protocolos: puede leer Modbus y publicar en MQTT, exponer un servidor OPC UA, o generar tráfico sintético continuo para capturar. En el laboratorio ocupa el papel de la “pasarela” en el ejercicio de DMZ y de consumidor MQTT del ESP32.

2.8. Herramientas de análisis y seguridad

2.8.1. Wireshark, tshark y tcpdump

Wireshark es el analizador de protocolos de referencia; `tshark` es su versión de línea de comandos y `tcpdump` la utilidad clásica de captura a archivo `.pcap`. Wireshark incluye disectores para todos los protocolos del laboratorio: `mbtcp/modbus`, `opcua`, `mqtt`, `s7comm`, `enip/cip`, `pn_io/pn_dcp`. Dos herramientas de Wireshark son especialmente útiles en OT: **Statistics** → **Conversations**, que resume quién habla con quién y cuánto, y **Statistics** → **I/O Graph**, que muestra paquetes por segundo y revela la periodicidad del sondeo. Cuando un protocolo va por un puerto no estándar (OPC UA en 50000), se indica con **Analyze** → **Decode As**.

2.8.2. netshoot

`nicolaka/netshoot` es una imagen Docker con decenas de herramientas de red (`tshark`, `tcpdump`, `nmap`, `iperf`, `socat`, `dig`...). Su uso más potente es `docker run -net container:<nombre>`: el contenedor auxiliar se une al *namespace de red* del contenedor objetivo y comparte su `eth0`, de modo que `tshark -i eth0` captura exactamente lo que ese servicio envía y recibe, sin tocar el host. Funciona igual en Linux, macOS y Windows, por lo que es el método de captura principal de esta guía.

2.8.3. nmap y scripts NSE

`nmap` descubre hosts y puertos; con el *Nmap Scripting Engine* incorpora scripts específicos de ICS: `modbus-discover` (identifica esclavos y lee su ID de dispositivo), `s7-info` (extrae modelo, firmware y número de serie de un S7 vía S7comm), `enip-info`, `bacnet-info`. El escaneo es la primera fase de cualquier intrusión y también la forma de inventariar una red propia; en OT debe hacerse con cuidado porque algunos PLC antiguos se bloquean ante escaneos agresivos.

2.8.4. Conpot

`Conpot` (proyecto Honeynet) es un **honeypot** de sistemas de control: un servidor que imita un PLC (por defecto un Siemens S7-300 con S7comm en el 102, Modbus en el 502, SNMP en el 161 y una web de diagnóstico en el 80) y registra en detalle cada conexión. Desplegado en una red real, cualquier tráfico que reciba es por definición sospechoso, porque ningún proceso legítimo lo necesita. En el laboratorio sirve para ver cómo “luce” un PLC ante `nmap` y para correlacionar un escaneo con el log del honeypot, que es el principio de la detección de intrusos en OT.

2.8.5. UaExpert

`UaExpert` (Unified Automation) es el cliente OPC UA gráfico de referencia, gratuito con registro. Permite descubrir endpoints, elegir política y modo de seguridad, gestionar certificados, navegar el espacio de direcciones, leer, escribir, suscribirse y llamar métodos. Es la forma más rápida de comparar una conexión sin seguridad con una cifrada.

2.8.6. Monitorización pasiva y port mirroring

Los sensores de red OT comerciales (Nozomi, Claroty, Dragos) y los de código abierto (Zeek, Suricata con reglas Modbus) no se instalan en los PLC: escuchan una copia del tráfico obtenida de un puerto espejo (*SPAN/mirror*) del switch o de un TAP. El laboratorio reproduce esa posición con un switch gestionable barato cuyo puerto espejo alimenta una NIC de captura del host (figura 2.8). La lección es que la visibilidad de la red depende de dónde se coloque el sensor, no solo de la herramienta.

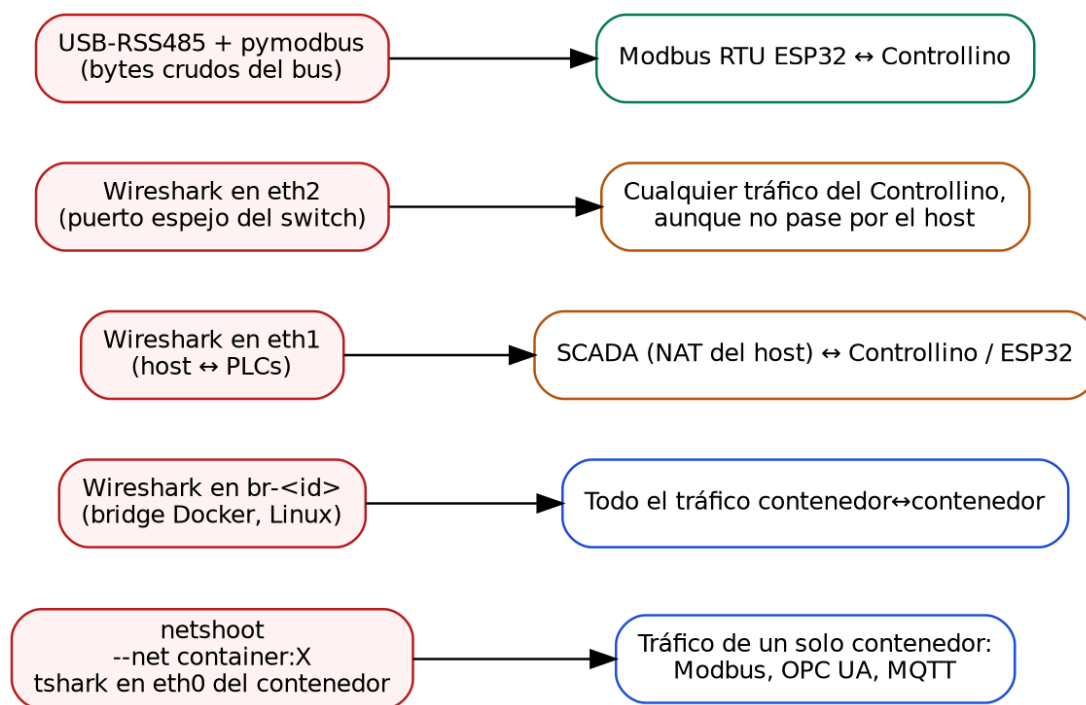


Figura 2.8: Puntos de captura del laboratorio y qué tráfico se observa en cada uno.

2.9. Hardware físico

2.9.1. Controllino

Controllino es una familia de PLC industriales basados en Arduino (ATmega328 en el MINI; ATmega2560 en MAXI y MEGA) con entradas y salidas a 24 V, relés, RTC y, en los modelos MAXI/MEGA, Ethernet 10/100 (controlador Wiznet W5100) y RS485. Se programa con el Arduino IDE y las librerías estándar (*Ethernet*, *ArduinoModbus*, *ArduinoRS485*), o con OpenPLC Editor en IEC 61131-3. Sus limitaciones son didácticamente valiosas: el W5100 admite solo 4 sockets simultáneos y el microcontrolador a 16 MHz responde a Modbus con una latencia claramente mayor que la de un simulador, lo que se mide en Wireshark.

2.9.2. ESP32 PLC 14 (Industrial Shields)

El ESP32 PLC 14 lleva un ESP32 de doble núcleo a 240 MHz con WiFi y Bluetooth, Ethernet RJ45, RS485 half-duplex, I2C de expansión, RTC, entradas analógicas 0–10 V / 4–20 mA y un relé. Industrial Shields proporciona el paquete de placas *industrialshields-esp32* y la librería **Tools40** con clases Modbus TCP/RTU maestro y esclavo. Su potencia y su doble conectividad (cable y WiFi) lo hacen ideal para tres papeles: esclavo Modbus TCP, **pasarela RTU→TCP** que expone al Controllino por Ethernet, y publicador MQTT.

2.9.3. RS485 y Modbus RTU

RS485 es una capa física serie diferencial (par A/B) multipunto y half-duplex, con alcance de hasta 1200 m y hasta 32 nodos por segmento sin repetidores. Requiere terminación de 120 Ω en ambos extremos y una referencia de masa común. Sobre ella corre Modbus RTU, donde el tiempo de silencio entre tramas es parte del protocolo: un analizador debe capturar los bytes crudos (adaptador USB-RS485 en paralelo al bus) y reconstruir las tramas por tiempo o por CRC. Wireshark no lee puertos serie directamente, pero un volcado se puede encapsular con *socat* y decodificar como *mbrtu*.

2.10. Ethernet industrial en tiempo real

PROFINET, EtherNet/IP con I/O implícito y EtherCAT trabajan en capa 2: usan tramas Ethernet propias (EtherType específico), multicast y tiempos de ciclo del orden de milisegundos. El bridge de Docker es una abstracción de capa 3 y en macOS/Windows ni siquiera expone la NIC real, por lo que estos protocolos no se simulan bien en contenedores. La tabla 2.3 resume las opciones si se quieren añadir al laboratorio.

Tabla 2.3: Opciones para estudiar Ethernet industrial de tiempo real fuera de Docker.

Protocolo	Opción recomendada	Notas
PROFINET (controlador)	TIA Portal + PLCSIM Advanced (Windows, trial)	Expone un controlador PROFINET virtual sobre la NIC del PC; filtro <code>pn_dcp</code> para ver el descubrimiento.
PROFINET (dispositivo)	p-net (RT-Labs) en Raspberry Pi o VM Linux	Puede correr en contenedor solo en Linux nativo con <code>network_mode: host</code> y <code>cap_add: NET_ADMIN, NET_RAW</code> .
EtherNet/IP	OpENer (adaptador open source) o driver de Ignition contra un PLC real	OpENer funciona en contenedor Linux con <code>network_mode: host</code> .
EtherCAT	SOEM + un esclavo real (p. ej. Beckhoff EL1008 usado)	No existe simulador serio sin hardware.

Con el Controllino y el ESP32 se cubren Modbus TCP/RTU y MQTT sobre hardware real; un S7-1200 o un Micro820 de segunda mano completan el laboratorio para PROFINET y EtherNet/IP.

Capítulo 3

Licencias y costes

Casi todo el laboratorio es software libre o gratuito. Las excepciones relevantes son Ignition, cuyo modelo conviene entender antes de usarlo en clase, y algunas herramientas de escritorio que exigen registro. La tabla 3.1 resume la situación; las secciones siguientes detallan los casos con matices.

Tabla 3.1: Licencia y coste de cada componente del laboratorio.

Componente	Licencia	Coste y condiciones
Docker Engine (Linux)	Apache 2.0	Gratuito.
Docker Desktop (macOS/Win)	Propietaria (Docker Subscription Service Agreement)	Gratuito para uso personal, educativo, proyectos de código abierto y empresas pequeñas; las grandes empresas requieren suscripción. Uso académico cubierto.
OpenPLC Runtime y Editor	GPLv3	Gratuito.
oits/modbus-server	Código abierto	Gratuito.
OPC PLC (Microsoft)	MIT	Gratuito.
asynqua (opcua-asynco)	LGPL-3.0	Gratuito.
Ignition	Propietaria	Trial gratuito ilimitado en bloques de 2 h reinicia- bles; Maker Edition gratuita para uso personal/e- ducativo no comercial; licencia comercial perpetua por servidor. Ver sección 3.1.
FUXA	MIT	Gratuito.
Node-RED	Apache 2.0	Gratuito.
Eclipse Mosquitto	EPL-2.0 / EDL-1.0	Gratuito.
Conpot	GPLv2	Gratuito.
netshoot, nmap, tcpdump	Código abierto (nmap: NPSL)	Gratuito.
Wireshark	GPLv2	Gratuito.
UaExpert	Propietaria	Gratuito para uso no comercial; requiere registro en unified-automation.com para descargar.
Arduino IDE	AGPL-3.0 / GPL	Gratuito.
Tools40 (Industrial Shields)	LGPL	Gratuito.
Factory I/O (opcional)	Propietaria	Simulador 3D de planta con Modbus/OPC UA. Ver- sión de prueba de 30 días; licencias de pago con descuento educativo. Solo Windows.

3.1. Ignition

Ignition no exige pago para usarse en laboratorio. Existen tres vías, resumidas en la figura 3.1.

3.1.1. Modo trial (edición Standard)

Al instalar Ignition sin licencia, *todos* los módulos funcionan de forma completa durante 2 horas. Al expirar, los módulos se detienen (el Gateway sigue vivo) y en la cabecera de la web aparece **Reset Trial**: un clic reinicia el contador otras 2 horas, sin límite de repeticiones, sin cuenta y sin conexión a Internet. Es el modo que usa el `docker-compose.yml` del laboratorio (IGNITION_EDITION: "standard") y el recomendado para las sesiones de clase, porque no impone restricciones de módulos y no depende de licencias externas. El único inconveniente es tener que pulsar **Reset Trial** en sesiones largas.

3.1.2. Maker Edition

Maker Edition es una licencia gratuita permanente destinada a uso **personal y educativo no comercial**. Se obtiene creando una cuenta en account.inductiveautomation.com, que entrega una clave de licencia y un token de activación; en Docker se pasan como IGNITION_EDITION: "maker", IGNITION_LICENSE_KEY e IGNITION_ACTIVATION_TOKEN. El Gateway debe poder contactar periódicamente con licensing.inductiveautomation.com (HTTPS); si pierde la conexión durante un tiempo vuelve al modo trial.

Sus limitaciones respecto a Standard son: máximo 10 sesiones Perspective simultáneas, 10 000 tags, sin Perspective Workstation (aplicación de escritorio), sin Vision (cliente Java clásico) y redundancia solo en modo independiente. Incluye los módulos que el laboratorio necesita: driver Modbus TCP, OPC UA (cliente y servidor), Perspective, Historian, Alarm Notification, Reporting, SQL Bridge, SFC, Serial Support, UDP/TCP Driver, Web Dev y los drivers Siemens, Allen-Bradley y Omron. Los módulos MQTT de Cirrus Link (MQTT Engine, Transmission, Distributor) son de terceros y están habilitados para Maker. En resumen, **no quita nada relevante para trabajar con Controllino, ESP32, OpenPLC ni con los servidores OPC UA del laboratorio**.

3.1.3. Licencias educativas y comerciales

La documentación de Maker indica que para instituciones educativas (aulas, proyectos de fin de carrera) Inductive Automation ofrece opciones específicas a través de su equipo de ventas. Para un uso formal en la Universidad de Cuenca lo correcto es solicitar esa licencia educativa. La licencia comercial es perpetua por servidor (no una suscripción) y solo tiene sentido en producción.

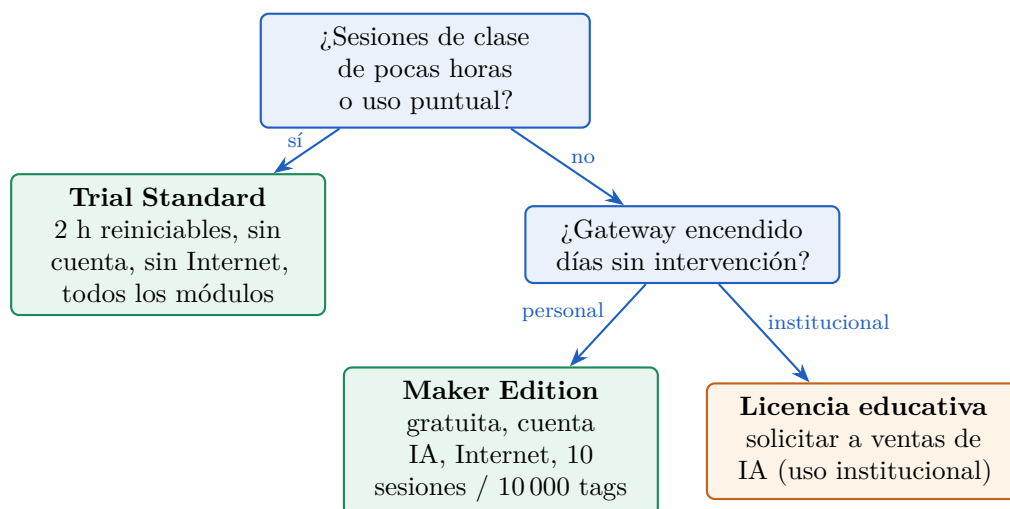


Figura 3.1: Elección de la vía de licenciamiento de Ignition para el laboratorio.

Recomendación adoptada

El laboratorio se despliega en modo **trial Standard**. Si se quiere un Gateway permanente para demostraciones, cambiar a Maker es un cambio de tres variables de entorno en el `docker-compose.yml`; el proyecto y los tags se conservan en el volumen `ignition-data`.

3.2. Docker Desktop

Docker Engine en Linux es software libre sin restricciones. Docker Desktop (el paquete gráfico para macOS y Windows que incluye la máquina virtual, Compose y Kubernetes) es gratuito para uso personal, educativo y de código abierto, y para empresas de menos de 250 empleados y menos de 10 millones de dólares de facturación; por encima de ese umbral requiere una suscripción de pago. El uso académico en la universidad está cubierto por la modalidad gratuita.

3.3. UaExpert y otras herramientas de escritorio

UaExpert se descarga gratis tras registrarse en la web de Unified Automation; la licencia permite uso no comercial y evaluación. No hay límite temporal. Wireshark, nmap, Arduino IDE y OpenPLC Editor son software libre. Factory I/O, cuyo instalador está en la carpeta del curso, es un simulador 3D de planta (Windows) que se comunica por Modbus TCP y OPC UA y puede sustituir al `modbus-sim` como “proceso” visual; su versión de prueba dura 30 días y las licencias educativas son de pago con descuento. Es opcional y no forma parte de las prácticas de esta guía.

Capítulo 4

Instalación y puesta en marcha

Este capítulo lleva desde un equipo sin nada instalado hasta los nueve contenedores en marcha y verificados. Recoge además los cuatro problemas que aparecieron al desplegar el laboratorio en macOS y cómo se resolvieron, porque cada uno ilustra un concepto de Docker que conviene conocer.

4.1. Requisitos

Tabla 4.1: Requisitos de software y hardware.

Herramienta	Dónde corre	Para qué
Docker Engine + Compose v2 (Linux) o Docker Desktop (macOS/Windows)	Host	Toda la infraestructura
Wireshark	Host	Analizar capturas
UaExpert	Host	Cliente OPC UA gráfico
Python 3.10+ con <code>pymodbus</code> y <code>asyncua</code> (opcional)	Host	Scripts propios; todo puede hacerse también desde contenedores
Arduino IDE 1.8.x/2.x + paquete <code>industrialshields-esp32</code> + placas Controllino	Host	Firmware de los PLC físicos
OpenPLC Editor	Host	Programar en IEC 61131-3 y subir firmware Modbus a las placas
Switch Ethernet, idealmente gestionable con <i>port mirroring</i> (p. ej. TP-Link TL-SG105E)	Físico	Segmento OT real y captura entre PLC
Adaptador USB-Ethernet (segunda NIC) y adaptador USB-RS485	Físico	Segmento OT aislado y análisis de Modbus RTU
Fuente 24 Vdc ≥ 2 A	Físico	Alimentar ambos PLC (no solo por USB)

Recursos mínimos del host: 8 GB de RAM (Ignition consume 1–2 GB; en macOS asigne al menos 6 GB a Docker Desktop), 15 GB de disco para imágenes y conexión a Internet para la primera descarga (la imagen de Ignition supera 1 GB).

4.2. Elección de plataforma

El laboratorio funciona en Linux, macOS y Windows, pero no de forma idéntica (sección 2.2). En **Linux nativo** el bridge de Docker es una interfaz visible en Wireshark, se puede usar

network_mode: host y macvlan, y el host alcanza las IPs 172.28.0.x. En **macOS y Windows** Docker Desktop ejecuta los contenedores dentro de una VM: se trabaja por puertos publicados y se captura desde dentro con netshoot. Ambas vías cubren todas las prácticas; la de Linux ofrece más puntos de captura.

Recomendación

Para un curso, la combinación más cómoda es Docker Desktop en el portátil de cada estudiante (macOS/Windows) para las prácticas de protocolos y SCADA, y un PC o VM Linux en el laboratorio físico, con la segunda NIC y el switch gestionable, para las prácticas de captura avanzada y hardware.

4.3. Instalación de Docker

4.3.1. macOS (Docker Desktop)

1. Descargar Docker Desktop desde <https://www.docker.com/products/docker-desktop/> eligiendo *Apple Silicon* o *Intel* según el equipo, e instalarlo arrastrándolo a Aplicaciones.
2. Abrirlo y aceptar el acuerdo de servicio. Esperar a que el icono de la barra de menús indique *Docker Desktop is running*.
3. En **Settings** → **Resources**: asignar al menos 6 GB de memoria y 4 CPU.
4. En **Settings** → **General**: en Apple Silicon, activar *Use Rosetta for x86_64/amd64 emulation on Apple Silicon*. Acelera notablemente las dos imágenes que solo existen para **amd64** (OpenPLC y Conpot).
5. Verificar en un terminal:

```
docker --version          # Docker version 2x.y
docker compose version    # Docker Compose version v2.x
docker run --rm hello-world
```

4.3.2. Linux (Ubuntu/Debian)

```
# Repositorio oficial de Docker
sudo apt-get update && sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/
    docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
    https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
    $VERSION_CODENAME) stable" \
    | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
# Usar docker sin sudo (cerrar sesión y volver a entrar después)
sudo usermod -aG docker $USER
```

4.3.3. Windows

Docker Desktop con el backend WSL 2 (instalar antes WSL con `wsl -install` en PowerShell como administrador y reiniciar). El comportamiento es el mismo que en macOS (VM oculta, acceso por puertos publicados). Wireshark puede instalarse en Windows para abrir los `.pcap` generados con netshoot. El laboratorio incluye un archivo específico, `docker-compose.windows.yml`, que se usa con la opción `-f`:

```
docker compose -f docker-compose.windows.yml up -d
docker compose -f docker-compose.windows.yml ps
```

Sus diferencias respecto a la versión macOS responden a tres particularidades de Windows:

- **Volúmenes con nombre** también para FUXA y Node-RED (no solo Ignition). Los bind mounts sobre NTFS (C:\Users\...) son lentos y no conservan el uid 1000 que Node-RED necesita. Solo se montan como bind mount los dos archivos de configuración (`server.json`, `mosquitto.conf`).
- **Fin de línea LF**. Git en Windows convierte por defecto los archivos a CRLF; Mosquitto no arranca con un `mosquitto.conf` en CRLF. El repositorio incluye un `.gitattributes` que fuerza LF; si el problema aparece, ejecutar `git config core.autocrlf false` y volver a clonar.
- **Puertos reservados**. Hyper-V y WSL reservan rangos de puertos dinámicos (habitualmente dentro de 49152–65535) y el 50000 del OPC PLC puede caer en uno de ellos; el error es *bind: An attempt was made to access a socket in a way forbidden by its access permissions*. Comprobar con `netsh interface ipv4 show excludedportrange protocol=tcp` y, si está reservado, cambiar el mapeo a "4850:50000" y usar `opc.tcp://localhost:4850` en UaExpert.

Recomendación adicional: clonar el proyecto dentro del sistema de archivos de WSL (`\\wsl$\\Ubuntu\\home\\<usuario>\\ot-lab`) y ejecutar los comandos desde el terminal de Ubuntu; el rendimiento de E/S es muy superior al de `/mnt/c`.

4.4. Estructura del proyecto

La carpeta `ot-lab` contiene todo lo necesario. Si se parte de cero:

```
mkdir -p ot-lab/{modbus,opcua,mosquitto,fuxa-data,nodered-data,captures,docs}
cd ot-lab
```

```
ot-lab/
|-- docker-compose.yml           # los 9 servicios (version macOS; Linux en .bak)
|-- docker-compose.linux.yml.bak # version original para Linux
|-- README.md
|-- modbus/server.json          # registros del esclavo Modbus simulado
|-- opcua/Dockerfile            # imagen del servidor OPC UA en Python
|-- opcua/server.py
|-- mosquitto/mosquitto.conf    # broker MQTT sin autenticacion (laboratorio)
|-- fuxa-data/ nodered-data/    # datos persistentes (bind mounts)
|-- captures/                  # volcados .pcap generados con netshoot
'-- docs/                      # esta guia, figuras y fuentes LaTeX
```

Los archivos completos están en el anexo A. El `docker-compose.yml` define una red bridge 172.28.0.0/24 con IP fija por servicio, un volumen con nombre para Ignition y nueve servicios cuyos puertos publicados se recogen en la tabla 4.2.

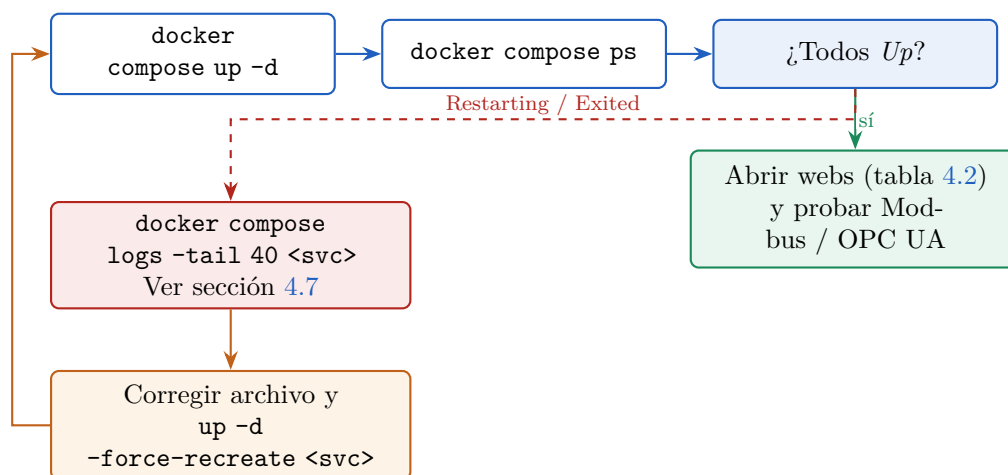
Tabla 4.2: Servicios del laboratorio, direcciones internas, puertos publicados y credenciales.

Servicio	IP interna	Puerto(s) host	Acceso	Credenciales
openplc	172.28.0.10	8080, 502	http://localhost:8080 Modbus :502	openplc / openplc
opc-plc	172.28.0.20	50000	opc.tcp://localhost:50000	anónimo
opcua-py	172.28.0.21	4840	opc.tcp://localhost:4840/1 ab/	anónimo
modbus-sim	172.28.0.30	5020	Modbus TCP :5020	—
ignition	172.28.0.50	8088	http://localhost:8088	admin / lab1234!
fuxa	172.28.0.51	1881	http://localhost:1881	—
node-red	172.28.0.52	1880	http://localhost:1880	—
mosquitto	172.28.0.60	1883	MQTT :1883	anónimo
conpot	172.28.0.90	10502, 10102, 10080, 10161/udp	http://localhost:10080	—

4.5. Arranque

```
cd ot-lab
docker compose up -d          # descarga/construye imagenes y arranca los 9 servicios
docker compose ps             # todos deben mostrar "Up" (Ignition tarda 1-3 min)
docker compose logs -f ignition # seguir un servicio; Ctrl+C para salir
```

La primera ejecución descarga unas 3 GB de imágenes y construye `opcua-py` desde su Dockerfile; tarda varios minutos. Las siguientes arrancan en segundos. La figura 4.1 resume la secuencia de verificación.

**Figura 4.1:** Secuencia de arranque y verificación del laboratorio.

4.5.1. Comprobaciones rápidas

```
# Estado (todos Up; node-red ademas "healthy")
docker compose ps

# Modbus: leer 3 holding registers del simulador -> [1234, 25, 72]
docker run --rm --network ot-lab python:3.12-slim sh -c "pip -q install pymodbus &&
python -c \"
```

```

from pymodbus.client import ModbusTcpClient
c=ModbusTcpClient('172.28.0.30',port=5020); c.connect()
print(c.read_holding_registers(0,count=3,slave=1).registers)\n"

# OPC UA: leer Tanque1.Nivel del servidor Python
docker run --rm --network ot-lab python:3.12-slim sh -c "pip -q install asyncua &&
python -c \"
import asyncio
from asyncua import Client
async def m():
    async with Client('opc.tcp://172.28.0.21:4840/lab/') as c:
        n=await c.nodes.root.get_child(['0:Objects','2:Tanque1','2:Nivel']); print(
        await n.read_value())
asyncio.run(m())\"

# Honeypot: la web falsa de Conpot responde
curl -s http://localhost:10080 | head -5

```

Resultado esperado

docker compose ps muestra los nueve contenedores en estado *Up*; la lectura Modbus devuelve [1234, 25, 72]; la lectura OPC UA devuelve un número entre 20 y 80; <http://localhost:8088> muestra la pantalla de bienvenida de Ignition.

4.6. Primer acceso a Ignition

Al abrir <http://localhost:8088> por primera vez, Ignition ya tiene el usuario **admin** con la contraseña del compose y ha aceptado el EULA por las variables de entorno. Aparece el diálogo **Enable Quick Start**, que ofrece crear una aplicación de ejemplo con un simulador de dispositivos interno, tags de muestra, historiador y diario de alarmas.

Elegir “No thanks, I’d like to start from scratch”

El Quick Start genera conexiones y tráfico internos que no pasan por la red *ot-lab* y confunden las capturas. El objetivo del laboratorio es que cada conexión que aparezca en Wireshark la haya creado el estudiante: partir de un Gateway vacío. Si se quiere ver la demo más adelante, se activa desde **Config** → **System** → **Quick Start**.

Tras entrar, la cabecera muestra el contador del modo trial (**Trial Mode 1:59:xx**). Cuando llegue a cero, los módulos se detienen y aparece el botón **Reset Trial**; pulsarlo devuelve otras 2 horas (capítulo 3). La configuración de dispositivos y OPC UA se describe en la sección 5.3.1.

4.7. Problemas encontrados en la puesta en marcha y su solución

Al desplegar el laboratorio en un Mac aparecieron cuatro fallos. Se documentan porque volverán a aparecer en cualquier equipo nuevo y porque cada uno enseña algo sobre cómo funcionan las imágenes.

4.7.1. Imágenes solo amd64 en Apple Silicon

Síntoma. `exec format error` o contenedor que se reinicia sin logs útiles. **Causa.** `tut-tas/openplc_v3` y `honeynet/conpot` solo publican variante `linux/amd64` (se comprobó en el manifiesto de Docker Hub); las otras siete imágenes son multi-arquitectura. **Solución.** Añadir `platform: linux/amd64` a esos dos servicios y activar Rosetta en Docker Desktop. En un PC Intel la línea es inocua.

4.7.2. Conpot: exec: "conpot": executable file not found in \$PATH

Síntoma. `docker compose up` falla al crear el contenedor con ese mensaje. **Causa.** La guía original usaba `command: conpot --template default ...`. Al inspeccionar la configuración de la imagen `honeynet/conpot:latest` se vio que *no* define `ENTRYPOINT`, que su `CMD` es la ruta completa `/home/conpot/.local/bin/conpot ...` y que ese directorio no está en el `PATH`. Al sobrescribir `command`, Docker intentó ejecutar el nombre `conpot` y no lo encontró. Además el `CMD` original lleva `-f` (*foreground*); sin él el proceso se demoniza y el contenedor termina. **Solución.**

```
command: ["/home/conpot/.local/bin/conpot", "--template", "default",
          "--logfile", "/var/log/conpot/conpot.log", "-f", "--temp_dir", "/tmp"]
```

Lección. `command` sustituye al `CMD` de la imagen y se concatena al `ENTRYPOINT` si existe. Antes de sobrescribirlo conviene ver qué trae la imagen: `docker inspect <imagen> -format '{{.Config.Entrypoint}} {{.Config.Cmd}}'`.

4.7.3. modbus-sim: KeyError: 'logging'

Síntoma. Contenedor en *Restarting*; el log muestra un *traceback* de Python en `CONFIG["server"]["logging"]`. **Causa.** La imagen `oitec/modbus-server` cambió el esquema de su archivo de configuración: ahora exige las secciones `server.logging`, `persistence` y `metrics`, y eliminó `zeroMode`. El JSON de la guía original correspondía a una versión anterior. Al revisar el esquema se detectó un segundo problema latente: las claves de los registros son **offsets de protocolo** (0-based), de modo que `"40001": 1234` colocaba el valor en el offset 40001 y la lectura del offset 0 que propone la práctica habría devuelto ceros. **Solución.** Reescribir `modbus/server.json` con el esquema actual y claves `"0"`, `"1"`, `"2"` (anexo A.4). **Lección.** Fijar versiones de imagen (`oitec/modbus-server:1.4.0` en lugar de `latest`) evita que un laboratorio deje de funcionar de un semestre a otro; y la notación Modbus 4xxxx frente a offsets 0-based es una trampa real (sección 2.3).

4.7.4. Ignition: cp: cannot stat './data/gateway.xml_clean'

Síntoma. Contenedor en *Restarting*; el log repite “Creating gateway.xml” y el error de `cp`. **Causa.** El `compose` montaba la carpeta vacía `./ignition-data` como *bind mount* sobre `/usr/local/bin/ignition/data`, ocultando los archivos semilla que la imagen trae en ese directorio (sección 2.2.3). El script de arranque busca `gateway.xml_clean` para generar la configuración inicial y no lo encuentra. **Solución.** Usar un volumen con nombre, al que Docker copia el contenido de la imagen en el primer arranque:

```
volumes:
  ignition-data:
services:
  ignition:
    volumes:
      - ignition-data:/usr/local/bin/ignition/data
```

Los datos quedan en el volumen `ot-lab-ignition-data` (`docker volume ls`); para copia de seguridad: `docker run -rm -v ot-lab-ignition-data:/d -v $PWD:/b alpine tar czf /b/ignition-backup.tgz -C /d ..`

4.8. Operación diaria

```
docker compose stop      # detener, conservando contenedores
docker compose start     # reanudar
```

```
docker compose down                # eliminar contenedores y red (volumenes y bind
    mounts persisten)
docker compose down -v             # ADEMÁS borra el volumen de Ignition (empezar
    de cero)
docker compose up -d --build opcua-py # reconstruir tras editar opcua/server.py
docker compose restart fuxa        # reiniciar un servicio
docker compose logs --tail 50 conpot # últimas líneas de log
docker stats                       # CPU y memoria por contenedor
docker compose pull && docker compose up -d # actualizar imágenes (riesgo: cambios de
    esquema)
```

Puertos en el host

Nada más debe escuchar en 502, 1880, 1881, 1883, 4840, 5020, 8080, 8088 ni 50000 del host. Si un puerto está ocupado (`lsof -i :8080` en macOS/Linux), cambiar el lado izquierdo del mapeo en el compose, p. ej. "8081:8080".

Capítulo 5

Prácticas guiadas

Con el laboratorio en marcha, este capítulo recorre los protocolos y herramientas en el orden de la figura 1.1. La figura 5.1 muestra la topología completa, incluido el segmento físico que se añade en el capítulo 6; la figura 5.2 resume qué protocolo une a cada par de componentes.

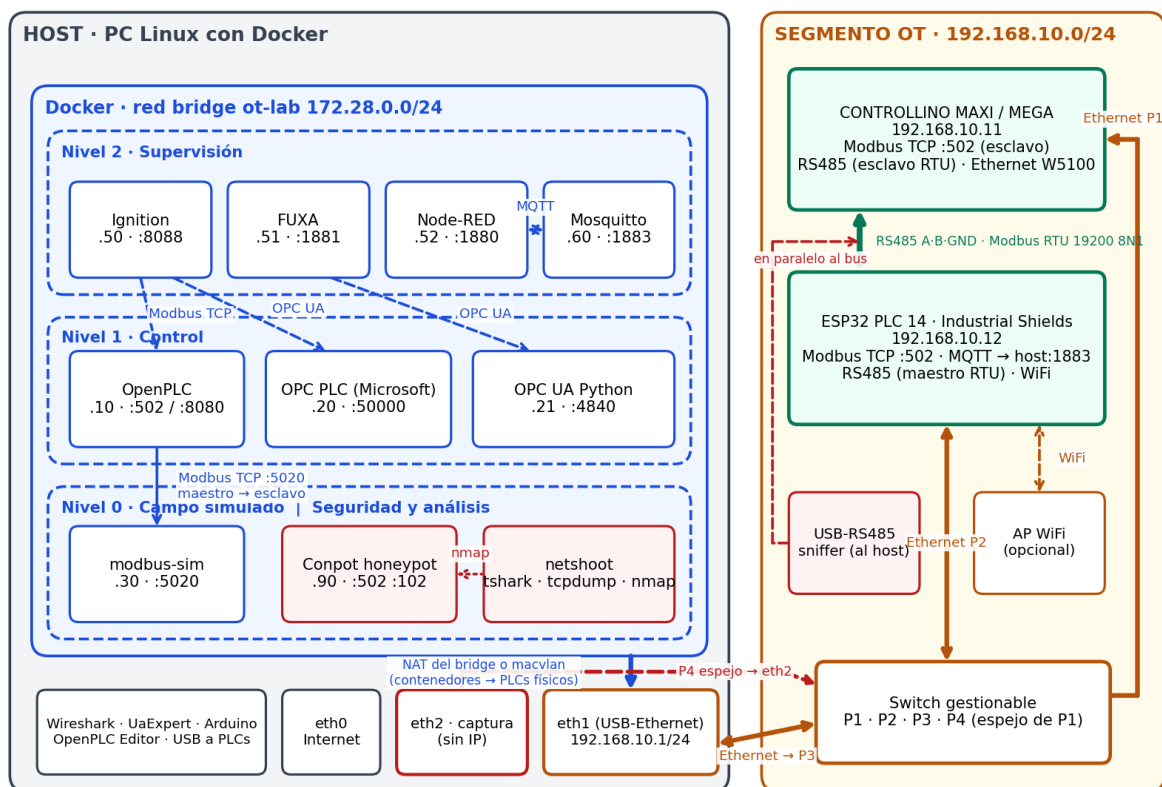


Figura 5.1: Topología completa del laboratorio: contenedores Docker en el host y segmento OT físico con Controllino y ESP32 PLC 14.

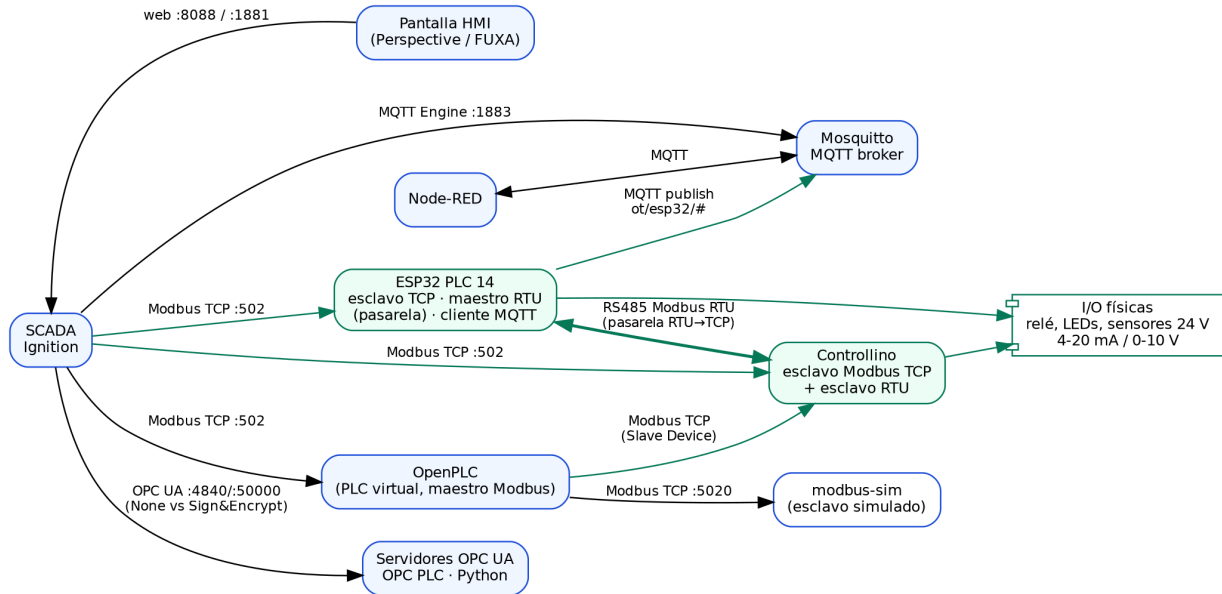


Figura 5.2: Flujo de datos y protocolos entre los componentes del laboratorio.

5.1. Práctica 1 — Modbus TCP

5.1.1. Esclavo simulado: modbus-sim

El servicio `modbus-sim` ejecuta `oitc/modbus-server`, un esclavo Modbus TCP en Python cuyos registros iniciales se definen en `modbus/server.json` (anexo A.4). Escucha en el puerto 5020 con estos valores:

Tabla 5.1: Registros iniciales del esclavo simulado.

Tabla	Offset	Valor	Notación 1-based
Holding registers	0, 1, 2	1234, 25, 72	40001–40003
Input registers	0	999	30001
Coils	0, 1	0, 1	00001–00002
Discrete inputs	0, 1	1, 0	10001–10002

```

docker compose up -d modbus-sim
docker compose logs modbus-sim      # "Start Modbus TCP server on 0.0.0.0:5020"

```

5.1.2. Cliente Modbus desde un contenedor

Sin instalar nada en el host, un contenedor Python unido a la red `ot-lab` lee y escribe registros:

```

docker run --rm --network ot-lab python:3.12-slim sh -c \
  "pip -q install pymodbus && python - <<'EOF'
from pymodbus.client import ModbusTcpClient
c = ModbusTcpClient('172.28.0.30', port=5020); c.connect()
print(c.read_holding_registers(0, count=3, slave=1).registers)
c.write_register(1, 42, slave=1)
print(c.read_holding_registers(0, count=3, slave=1).registers)
c.close()
EOF"

```

Resultado esperado

Primera línea [1234, 25, 72]; segunda [1234, 42, 72]. La escritura (FC 06) modificó el offset 1.

Alternativa con mbpoll: `docker run -rm -network ot-lab debian:bookworm sh -c "apt-get -qq update && apt-get -qq install -y mbpoll >/dev/null && mbpoll -m tcp -a 1 -r 1 -c 3 -p 5020 172.28.0.30 -1"`. Obsérvese que mbpoll usa direcciones 1-based (-r 1 es el offset 0).

5.1.3. PLC virtual: OpenPLC

1. Abrir <http://localhost:8080> (openplc/openplc).
2. **Programs** → **Upload**: subir un programa .st (el repositorio de OpenPLC incluye ejemplos; el más simple copia una entrada a una salida) y **Launch program. Start PLC**.
3. **Settings**: comprobar que *Modbus Server* está habilitado en el puerto 502. OpenPLC ya es un esclavo Modbus: sus %QX0.0 son coils 0–7, sus %IW100 registros de entrada, etc.
4. Para que además actúe como **maestro** y lea el simulador: **Slave Devices** → **Add new device**, protocolo *Generic Modbus TCP Device*, IP 172.28.0.30, puerto 5020, *Holding Registers* – *Read start* 0 size 3. OpenPLC mapea esos registros a %IW100..%IW102 y los sondea cada 100 ms: ya hay tráfico Modbus continuo entre dos contenedores.

Nota

OpenPLC, como maestro, abre una conexión TCP al simulador y la mantiene, enviando peticiones FC 03 periódicas. Esa es la primera “firma” de sondeo que se analizará en la práctica 5.4.

5.2. Práctica 2 — OPC UA**5.2.1. Servidor simulado: OPC PLC**

opc-plc arranca con opciones que generan nodos lentos (-sn=5 -sr=10: 5 nodos cada 10 s), rápidos (-fn=5 -fr=1) y con anomalías (-gn=5), acepta cualquier certificado de cliente (-autoaccept) y permite usuarios anónimos (-ut). Endpoint: `opc.tcp://localhost:50000` desde el host; `opc.tcp://172.28.0.20:50000` desde otros contenedores.

5.2.2. Servidor propio en Python

opcua-py se construye desde opcua/Dockerfile y ejecuta `server.py` (anexo A.5), que publica el objeto `Tanque1` con Nivel (senoide 20–80), Temperatura (15–25) y Valvula (booleano escribible). Endpoint: `opc.tcp://localhost:4840/lab/`. Para modificarlo: editar `server.py` y `docker compose up -d -build opcua-py`.

5.2.3. Cliente UaExpert

1. **Add Server** → **Custom Discovery**: introducir `opc.tcp://localhost:50000` y `opc.tcp://localhost:4840/lab/`. UaExpert lista los endpoints de cada servidor con sus políticas.
2. Conectar primero con **None / None** (sin seguridad) y anónimo. Navegar hasta **Objects** → **Tanque1** → **Nivel**, arrastrarlo a *Data Access View* y ver cómo cambia cada segundo. Escribir `true` en **Valvula**.
3. Mientras, capturar el tráfico (práctica 5.4) y observar en Wireshark, con filtro `opcua`, los mensajes `Hello`, `OpenSecureChannel`, `CreateSession`, `ActivateSession`, `Browse`, `Read`, `CreateSubscription` y los `Publish` periódicos, con los valores legibles.

- Desconectar y reconectar con **Basic256Sha256 / Sign & Encrypt**. Aceptar el certificado del servidor (*Trust Server Certificate*). Repetir la captura: tras el `OpenSecureChannel` solo se ven mensajes MSG con carga cifrada.

Resultado esperado

Dos capturas: una con valores de `Nivel` legibles en los `Publish`, otra donde solo los mensajes `HEL/ACK`, `OPN` y los certificados son visibles. Anotar qué campos siguen en claro en la segunda (URL del endpoint, política, certificados X.509).

Cliente en contenedor, como alternativa a UaExpert:

```
docker run --rm --network ot-lab python:3.12-slim sh -c \
  "pip -q install asyncua && python - <<'EOF'
import asyncio
from asyncua import Client
async def main():
    async with Client('opc.tcp://172.28.0.21:4840/lab/') as c:
        n = await c.nodes.root.get_child(['0:Objects', '2:Tanque1', '2:Nivel'])
        print('Nivel =', await n.read_value())
asyncio.run(main())
EOF"
```

5.3. Práctica 3 — SCADA

5.3.1. Ignition

Ignition corre *dentro* de Docker, por lo que debe usar las IPs de la red `ot-lab`, nunca `localhost`.

- Entrar en <http://localhost:8088> (admin/lab1234!); rechazar el Quick Start (sección 4.6).
- Config** → **OPC UA** → **Device Connections** → **Create new Device** → **Modbus TCP**: nombre `OpenPLC`, host `172.28.0.10`, puerto `502`. Repetir con nombre `ModbusSim`, host `172.28.0.30`, puerto `5020`. En cada dispositivo, pestaña *Advanced*: *Max concurrent requests* = 1 si se va a comparar con los PLC físicos.
- En el dispositivo Modbus, **More** → **Addresses**: definir un rango, p. ej. prefijo `HR`, tipo *HoldingRegister*, inicio 1, fin 3 (Ignition usa direcciones 1-based). Esto genera tags navegables.
- Config** → **OPC UA** → **Connections** → **Create new**: endpoint `opc.tcp://172.28.0.20:50000` y otro `opc.tcp://172.28.0.21:4840/lab/`, política `None` para empezar. Ignition también expone su propio servidor OPC UA en el 62541.
- Status** → **Connections** → **Devices** debe mostrar los dispositivos como *Connected*.
- Descargar el **Designer Launcher** desde la página principal del Gateway, añadir el Gateway `localhost:8088`, abrir el Designer, crear un proyecto *Perspective*, arrastrar desde el *Tag Browser* ([default] → dispositivos) los tags `HR1..HR3` y `Tanque1/Nivel` a una vista y guardar. Abrir la sesión *Perspective* en el navegador.

Resultado esperado

Los valores del simulador (1234, 25, 72) y el `Nivel` senoidal aparecen en la vista *Perspective* y se actualizan. Al capturar en `ignition` se ve un sondeo Modbus regular (por defecto cada 1 s, tag group *Default*) hacia cada dispositivo y un `Publish` OPC UA hacia cada servidor.

5.3.2. FUXA

En <http://localhost:1881>: **Connections** → **+**: tipo *ModbusTCP*, IP `172.28.0.30`, puerto `5020`, esclavo 1; añadir tags con dirección 40001–40003 (FUXA usa notación 1-based con prefijo). Otra conexión *OPC UA* a `opc.tcp://172.28.0.21:4840/lab/` y navegar hasta `Tanque1`. Crear

una vista con un *gauge* enlazado a Nivel. Comparar en Wireshark el patrón de sondeo de FUXA con el de Ignition: número de conexiones TCP y periodo.

5.3.3. Node-RED

En <http://localhost:1880>: **Manage palette** → **Install**: node-red-contrib-modbus y node-red-contrib-opcua. Flujo mínimo: nodo *Modbus Read* (servidor 172.28.0.30:5020, FC 3, dirección 0, cantidad 3, cada 500 ms) → *debug*. Segundo flujo: *OpcUa Client* suscrito a ns=2;s=... → *mqttn out* hacia 172.28.0.60 topic ot/nodered/nivel. Node-RED se convierte así en pasarela OPC UA→MQTT y en generador de tráfico continuo.

5.4. Práctica 4 — Captura y análisis de tráfico

5.4.1. Capturar dentro de la red Docker con netshoot

El método universal (Linux, macOS, Windows) es unir un contenedor *netshoot* al namespace de red del servicio a observar:

```
# Ver 30 s de trafico Modbus que llega a OpenPLC, decodificado
docker run --rm --net container:openplc nicolaka/netshoot \
  tshark -i eth0 -a duration:30 -f "tcp port 502" -Y modbus

# Guardar a pcap el trafico OPC UA del servidor Microsoft, para abrirlo en Wireshark
docker run --rm --net container:opc-plc -v "$PWD/captures:/cap" nicolaka/netshoot \
  tcpdump -i eth0 -w /cap/opcua.pcap "tcp port 50000"

# Todo lo que entra y sale de Ignition (sondeo a todos los dispositivos)
docker run --rm --net container:ignition -v "$PWD/captures:/cap" nicolaka/netshoot \
  tcpdump -i eth0 -w /cap/ignition.pcap "not port 8088"
```

Los archivos aparecen en *captures/* del host y se abren con Wireshark. Ctrl+C detiene la captura.

5.4.2. Capturar desde el host (solo Linux)

```
docker network inspect ot-lab -f '{{.Id}}' | cut -c1-12      # -> br-<id>
sudo wireshark -i br-<id> -k -f "tcp port 502 or tcp port 5020 or tcp port 50000 or tcp
port 4840"
```

5.4.3. Filtros útiles en Wireshark

Tabla 5.2: Filtros de visualización de Wireshark para el laboratorio.

Filtro	Qué muestra
mbtcp	Modbus TCP completo (MBAP + PDU)
modbus.func_code == 3	Lecturas de holding registers
modbus.func_code in {5 6 15 16}	Todas las escrituras (coils y registros)
modbus.exception_code	Respuestas de excepción
opcua	OPC UA (decodifica en claro solo con None/None)
opcua.transport.type == "HEL"	Inicio de conexiones OPC UA
opcua.servicenodeid.numeric == 631	Publish requests
mqtt	MQTT; mqtt.msgtype == 3 solo PUBLISH
s7comm	Tráfico S7 (contra Conpot)
tcp.analysis.retransmission	Retransmisiones (ejercicio WiFi)
ip.addr == 172.28.0.50	Todo lo que toca a Ignition

Statistics → **Conversations** resume quién habla con quién; **Statistics** → **I/O Graph** con el filtro `mbtcp` muestra el sondeo como una señal periódica: un SCADA leyendo cada segundo es una firma que conviene reconocer a simple vista. Si OPC UA en el 50000 no se decodifica: **Analyze** → **Decode As** → **TCP port 50000** → **OpcUa**.

5.4.4. Reconocimiento con nmap

```
# Inventario de la red del laboratorio
docker run --rm --network ot-lab nicolaka/netshoot \
  nmap -sT -p 502,5020,102,4840,50000,8080,1881,8088,1883 172.28.0.0/24

# Scripts NSE específicos de ICS
docker run --rm --network ot-lab nicolaka/netshoot \
  nmap -p 502 --script modbus-discover 172.28.0.10 172.28.0.30
```

Resultado esperado

nmap identifica nueve hosts activos y sus puertos; `modbus-discover` devuelve el *slave ID* y, en OpenPLC, el identificador del dispositivo. A partir solo de puertos y banners debe poder clasificarse cada host por función (ejercicio 4 del capítulo 7).

5.5. Práctica 5 — Honeypot ICS con Conpot

Conpot simula un Siemens S7-300 (S7comm en el 102, Modbus en el 502, SNMP en el 161 y web en el 80). Sus puertos están publicados en el host con prefijo 10 (10102, 10502, 10080, 10161) para no chocar con OpenPLC.

```
docker run --rm --network ot-lab nicolaka/netshoot \
  nmap -p 502,102 --script modbus-discover,s7-info 172.28.0.90
docker compose logs -f conpot
```

Resultado esperado

s7-info devuelve modelo, número de serie y versión de firmware “del PLC”; el log de Conpot registra cada conexión con IP de origen, protocolo y función solicitada, exactamente como lo haría un IDS. Abrir <http://localhost:10080> muestra la web de diagnóstico falsa.

Escribir después un registro Modbus en Conpot desde pymodbus (172.28.0.90, puerto 502) y correlacionar la entrada del log con la captura: es el ejercicio 6.

Capítulo 6

Práctica 6 — Integración de los PLC físicos

Aquí el laboratorio deja de ser simulado. El Controllino y el ESP32 PLC 14 se programan con Arduino IDE (o con OpenPLC Editor) y hablan Modbus, así que encajan como dispositivos de nivel 1/0 que Ignition, FUXA y el OpenPLC de Docker pueden leer y escribir.

6.1. Equipos

Tabla 6.1: Características de los dos PLC físicos y su papel en el laboratorio.

	Controllino	ESP32 PLC 14 (Industrial Shields)
MCU	ATmega2560 (MAXI/MEGA) o ATmega328 (MINI), 16 MHz	ESP32 doble núcleo 240 MHz
Ethernet	Sí en MAXI/MEGA (W5100, 10/100). MINI no tiene	Sí (RJ45)
Inalámbrico	No	WiFi + BLE
Serie industrial	RS485 en MAXI/MEGA	RS485 half-duplex
Otros	RTC, E/S 24 V, relés	I2C de expansión, RTC, E/S 24 V, 1 relé, entradas 0–10 V / 4–20 mA
Rol en el lab	Esclavo Modbus TCP “clásico”, cableado, lento y determinista	Esclavo Modbus TCP, pasarela RTU→TCP, publicador MQTT; comparar Ethernet frente a WiFi

Nota

Si el Controllino es **MINI** (sin Ethernet ni RS485), conectarlo por USB y usarlo como esclavo Modbus RTU sobre serie; el host o el ESP32 hacen de pasarela. El resto del capítulo asume MAXI o MEGA.

6.2. Topología física y direccionamiento

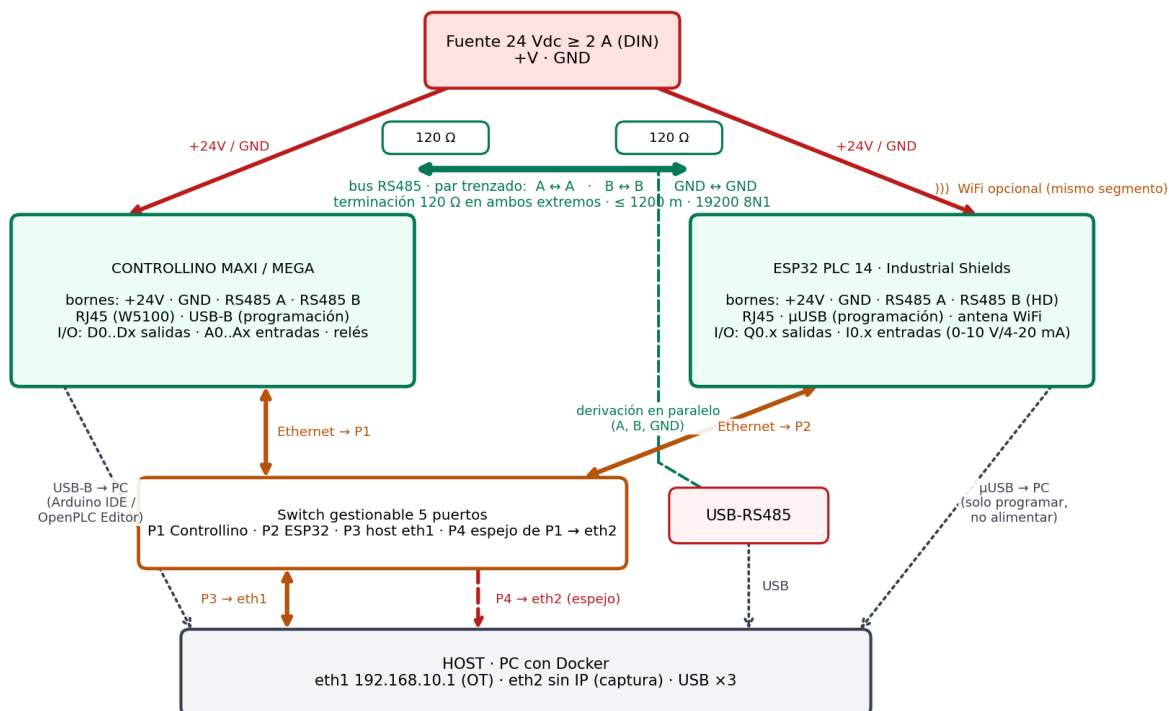


Figura 6.1: Conexiones físicas: alimentación, Ethernet, bus RS485 y USB de programación.

1. Conectar la segunda NIC del host (**eth1**, USB-Ethernet), el Controllino y el ESP32 a un switch. Alimentar ambos PLC a 24 V desde la fuente, no por USB.
2. IPs estáticas: host 192.168.10.1/24, Controllino .11, ESP32 .12. Sin puerta de enlace en los PLC: no tienen por qué salir a ningún sitio.
3. Configurar la NIC del host y comprobar conectividad:

```
# Linux
sudo ip addr add 192.168.10.1/24 dev eth1 && sudo ip link set eth1 up
# macOS: Ajustes -> Red -> adaptador USB -> Configurar IPv4: Manual, 192.168.10.1 / 255.255.255.0
ping -c 3 192.168.10.11 && ping -c 3 192.168.10.12
```

4. Si el switch es gestionable, configurar *port mirroring* del puerto del Controllino hacia un puerto donde haya una tercera NIC de captura (**eth2**, sin IP). Wireshark verá el tráfico SCADA $\leftrightarrow$ PLC aunque no pase por el host: es la posición de un sensor pasivo de red OT.

El segmento 192.168.10.0/24 va en una NIC dedicada, separado de la red doméstica o de la universidad: eso ya es un ejercicio de segmentación.

6.3. Cómo llegan los contenedores a los PLC

A) Bridge + NAT (por defecto; cualquier SO). Los contenedores salen por NAT del host, así que Ignition, FUXA y OpenPLC alcanzan 192.168.10.11:502 sin configurar nada. Limitación: los PLC no pueden iniciar conexiones hacia un contenedor salvo a través de un puerto publicado en el host (el ESP32 publica MQTT a 192.168.10.1:1883, que es el puerto publicado de Mosquitto). En Wireshark, todo el tráfico de los contenedores aparece con la IP del host.

B) macvlan (solo Linux nativo). Cada contenedor recibe una IP y una MAC propias en

192.168.10.0/24; en Wireshark aparecen como equipos distintos y se puede hacer nmap “desde fuera”.

```
networks:
  ot-phys:
    driver: macvlan
    driver_opts:
      parent: eth1
    ipam:
      config:
        - subnet: 192.168.10.0/24
          ip_range: 192.168.10.128/25 # rango reservado a contenedores
          gateway: 192.168.10.1
```

Añadir a ignition, fuxa, node-red y openplc una segunda red ot-phys: { ipv4_address: 192.168.10.150 } (y sucesivas). Con macvlan el host *no* puede hablar con sus propios contenedores por eth1; hace falta una interfaz puente:

```
sudo ip link add ot-shim link eth1 type macvlan mode bridge
sudo ip addr add 192.168.10.2/32 dev ot-shim && sudo ip link set ot-shim up
sudo ip route add 192.168.10.128/25 dev ot-shim
```

6.4. Firmware: opción rápida con OpenPLC Editor

OpenPLC Editor compila un programa IEC 61131-3 y lo sube directamente a placas Arduino, incluidos Controllino MAXI/MEGA/MINI y ESP32, dejando la placa como esclavo Modbus (TCP y/o RTU) con las E/S mapeadas a %IX, %QX, %IW, %QW.

1. Nuevo proyecto; escribir un ladder sencillo (p. ej. %QX0.0 := %IX0.0).
2. **Transfer program to PLC:** *Board* = Controllino MAXI o ESP32; pestaña **Modbus**: habilitar TCP, IP 192.168.10.11 (Controllino) o .12 (ESP32), puerto 502; en el ESP32 elegir Ethernet o WiFi (SSID/clave).
3. Compilar y subir. El dispositivo ya responde Modbus TCP.
4. En el OpenPLC de Docker (<http://localhost:8080>) **Slave Devices** → **Add**: Generic Modbus TCP, IP del PLC físico, mapear sus registros. El PLC virtual controla ahora E/S reales. En Ignition, añadir ambos como dispositivos Modbus TCP igual que el simulador.

Industrial Shields documenta el mapa de pines de sus placas; comprobar qué pines Arduino corresponden a I0.x/Q0.x antes de usarlos desde OpenPLC.

6.5. Firmware: opción con Arduino IDE (control total)

Instalar en Arduino IDE el paquete de placas `industrialshields-esp32` y la librería **Tools40** (clases `ModbusTCPSlave`, `ModbusTCPMaster`, `ModbusRTUSlave`, `ModbusRTUMaster`); para el Controllino, las placas Controllino y la librería **ArduinoModbus** (con `ArduinoRS485`).

ESP32 PLC 14 como esclavo Modbus TCP por Ethernet (esqueleto; partir de **File** → **Examples** → **Tools40** para el modelo exacto):

```
#include <Ethernet.h>
#include <ModbusTCPSlave.h>

byte mac[] = {0xDE,0xAD,0xBE,0xEF,0x00,0x12};
IPAddress ip(192,168,10,12);
ModbusTCPSlave modbus(502);

bool coils[4];
bool dinputs[4];
```

```

uint16_t hregs[4];
uint16_t iregs[4];

void setup() {
  Ethernet.begin(mac, ip);
  modbus.begin();
  modbus.setCoils(coils, 4);
  modbus.setDiscreteInputs(dinputs, 4);
  modbus.setHoldingRegisters(hregs, 4);
  modbus.setInputRegisters(iregs, 4);
}

void loop() {
  dinputs[0] = digitalRead(IO_0);
  iregs[0] = analogRead(IO_5);           // entrada analogica
  digitalWrite(Q0_0, coils[0]);         // el SCADA escribe el coil 0 -> sale por Q0.0
  modbus.update();
}

```

Controllino MAXI como esclavo Modbus TCP (ArduinoModbus):

```

#include <Controllino.h>
#include <SPI.h>
#include <Ethernet.h>
#include <ArduinoRS485.h>
#include <ArduinoModbus.h>

byte mac[] = {0xDE,0xAD,0xBE,0xEF,0x00,0x11};
EthernetServer server(502);
ModbusTCPServer modbusTCP;

void setup() {
  Ethernet.begin(mac, IPAddress(192,168,10,11));
  server.begin();
  modbusTCP.begin();
  modbusTCP.configureCoils(0, 8);
  modbusTCP.configureDiscreteInputs(0, 8);
  modbusTCP.configureHoldingRegisters(0, 8);
  pinMode(CONTROLLINO_D0, OUTPUT);
  pinMode(CONTROLLINO_A0, INPUT);
}

void loop() {
  EthernetClient client = server.available();
  if (client) {
    modbusTCP.accept(client);
    while (client.connected()) {
      modbusTCP.poll();
      modbusTCP.discreteInputWrite(0, digitalRead(CONTROLLINO_A0));
      digitalWrite(CONTROLLINO_D0, modbusTCP.coilRead(0));
    }
  }
}

```

Comprobar desde el host: `mbpoll -m tcp -a 1 -t 0 -r 1 -c 4 192.168.10.11` (leer coils) y `mbpoll -m tcp -a 1 -t 0 -r 1 192.168.10.11 1` (escribir el coil 0): debe oírse el relé o verse el LED de salida.

6.6. Modbus RTU sobre RS485 entre los dos PLC

Cablear $A \leftrightarrow A$, $B \leftrightarrow B$ y GND entre el RS485 del Controllino y el del ESP32, con terminación de $120\ \Omega$ en ambos extremos. Controllino como **esclavo RTU** (`ModbusRTUserver` de `ArduinoModbus`, 19200 8N1, ID 1); ESP32 como **maestro RTU** (`ModbusRTUMaster` de `Tools40`) que lee los registros del Controllino y los copia en sus propios *holding registers* Modbus TCP. Desde Ignition se lee el Controllino “a través” del ESP32, que actúa de **pasarela RTU→TCP**, exactamente lo que hacen los gateways Moxa o Anybus en planta.

Para analizar el bus serie: el adaptador USB-RS485 del host se conecta en paralelo al bus (A, B, GND) y se capturan los bytes crudos con `pymodbus` (`ModbusSerialClient`, `framer='rtu'`) o con un script que vuelque `serial.read()` en hexadecimal. Wireshark no lee serie directamente; se puede exponer el puerto por TCP con `socat` y decodificar como `mbrtu` con **Decode As**.

6.7. MQTT desde el ESP32

El broker Mosquitto ya está en el compose (`172.28.0.60:1883`, publicado en el host). En el ESP32, con la librería `PubSubClient`, conectar a `192.168.10.1:1883`, publicar cada segundo `ot/esp32/nivel` y suscribirse a `ot/esp32/cmd`. Node-RED (nodo *mqtt in*) e Ignition (módulo MQTT Engine) lo consumen. Ejercicio de red: MQTT viaja en claro sobre TCP 1883; capturar, leer el payload, y después habilitar un listener TLS en el 8883 de Mosquitto (certificado autofirmado, `cafile/certfile/keyfile` en `mosquitto.conf` y republicar el puerto) para ver la diferencia. Cambiar el ESP32 de Ethernet a WiFi y comparar el *jitter* en **Statistics** → **I/O Graph** con el filtro `mqtt`.

Capítulo 7

Ejercicios propuestos

Los ejercicios están ordenados de menor a mayor dificultad y cada uno se apoya en los anteriores. Para cada uno se espera una captura `.pcap` comentada, las respuestas a las preguntas y, cuando proceda, la configuración empleada.

7.1. Con el laboratorio en Docker

- E1. Anatomía Modbus.** Capturar una lectura (FC 03) y una escritura (FC 06) contra `modbus-sim`. Identificar en la MBAP el *transaction ID*, el *unit ID* y el campo *length*; en la PDU el código de función, la dirección y los datos. Provocar una excepción leyendo un registro fuera de rango (offset 70000 no es posible; usar el FC 04 sobre `openplc` en un offset alto) y explicar el código devuelto.
- E2. Polling.** Configurar en Ignition un *tag group* de 500 ms para el dispositivo OpenPLC y medir en I/O Graph los paquetes por segundo. Cambiar a 100 ms y comparar. ¿Cuántas peticiones genera Ignition por ciclo y por qué? ¿Cómo agrupa registros contiguos?
- E3. OPC UA en claro frente a cifrado.** Dos capturas de UaExpert (None/None frente a Basic256Sha256 Sign&Encrypt). Explicar qué sigue siendo visible (endpoints, políticas, certificados, tamaños y periodicidad) y qué deja de serlo. ¿Qué podría inferir un atacante pasivo de la sesión cifrada?
- E4. Reconocimiento.** Con `nmap` sobre 172.28.0.0/24 y sin consultar el compose, clasificar cada host por función solo a partir de puertos y banners. Después ejecutar `modbus-discover` y `s7-info` y anotar qué información adicional exponen los protocolos ICS sin autenticación.
- E5. Segmentación.** Crear una segunda red `ot-dmz` en el compose, mover Ignition a ella y dejar Node-RED conectado a ambas como pasarela. Verificar con `nmap` desde Ignition que ya no ve a OpenPLC directamente, y montar en Node-RED un flujo que le “publique” los datos por OPC UA o MQTT. Discutir qué se gana y qué se pierde.
- E6. Detección.** Lanzar un escaneo y una escritura Modbus no autorizada contra Conpot. Correlacionar cada línea del log del honeypot con los paquetes de la captura. Proponer una regla de alerta (p.ej. Suricata: cualquier FC de escritura hacia el 502 desde una IP distinta del SCADA).
- E7. Manipulación.** Desde `pymodbus` escribir un coil de OpenPLC mientras Ignition lo muestra en pantalla. Observar el efecto en la HMI y localizar la escritura en la captura. ¿Cómo distinguiría un operador una escritura legítima del SCADA de una externa, si ambas son idénticas en el cable?

7.2. Con los PLC físicos

- E8. Simulado frente a real.** Ignition sondea a la vez `modbus-sim`, OpenPLC (Docker) y el Controllino. En Wireshark comparar el *delta time* petición→respuesta de cada uno (columna `modbus.response_time` o `tcp.time_delta`). El Controllino (W5100, AVR a 16 MHz) será visiblemente más lento. ¿Qué implica para el periodo mínimo de sondeo del SCADA?
- E9. Port mirroring.** Capturar el tráfico Ignition ↔ Controllino desde la NIC conectada al puerto espejo del switch, sin tocar el host. Comparar con la captura desde `eth1`: ¿qué tráfico aparece en una y no en la otra?
- E10. Ethernet frente a WiFi.** Mismo firmware en el ESP32 por cable y por WiFi; medir latencia, *jitter* y retransmisiones (`tcp.analysis.retransmission`). Añadir interferencia (microondas, descargas en la misma WiFi) y repetir. Concluir sobre el uso de WiFi en control.
- E11. Pasarela RTU→TCP.** Implementar la sección 6.6 y observar que una sola petición Modbus TCP al ESP32 desencadena tráfico RS485 hacia el Controllino. Medir el tiempo extra que añade la pasarela.
- E12. Segmentación real.** Con `nftables` (Linux) en el host permitir solo TCP 502 desde la IP de Ignition hacia `192.168.10.0/24`. Verificar con `nmap` desde otro contenedor que ya no alcanza los PLC; intentar la escritura de coil desde un equipo de la red doméstica y comprobar que falla.
- E13. Escritura no autorizada con efecto físico.** Desde un tercer host escribir el coil del relé del Controllino o del ESP32 y verlo conmutar. Diseñar una regla de detección (Zeek o Suricata con reglas Modbus) que alerte de escrituras desde IPs distintas del SCADA y comprobar que dispara.
- E14. MQTT en claro frente a TLS.** Sección 6.7: comparar capturas en 1883 y 8883. ¿Qué metadatos siguen visibles con TLS (tamaño, periodicidad, SNI)?

Entregable sugerido por ejercicio

Archivo `.pcap` (o `.pcapng`) con un filtro de visualización guardado, una captura de pantalla de Wireshark con los campos relevantes señalados, y un párrafo de conclusiones. Para los ejercicios de configuración, el fragmento de `docker-compose.yml` o de firmware modificado.

Capítulo 8

Problemas frecuentes

Los cuatro fallos de la puesta en marcha (Conpot, modbus-sim, Ignition, imágenes amd64) se describen con detalle en la sección 4.7. Esta lista recoge el resto de situaciones habituales.

Tabla 8.1: Problemas frecuentes y solución.

Síntoma	Causa y solución
Un contenedor queda en <i>Restarting</i>	Ver su log: <code>docker compose logs -tail 40 <servicio></code> . Casi siempre es un archivo de configuración montado con formato incorrecto o un cambio de esquema tras <code>pull</code> . Fijar la etiqueta de versión de la imagen.
Puerto 502 ocupado o requiere privilegios	En Linux, mapear "1502:502" si no se quiere correr Docker como root o algo ya escucha en 502. En macOS Docker Desktop publica puertos bajos sin problema.
<code>exec format error</code>	Imagen amd64 en host ARM. Añadir <code>platform: linux/amd64</code> o construir la imagen localmente.
Ignition muy lento en Apple Silicon	La imagen de Ignition es multi-arquitectura y corre nativa; la lentitud viene de memoria escasa. Asignar ≥ 6 GB a Docker Desktop. OpenPLC y Conpot sí van emulados: activar Rosetta.
UaExpert no conecta a <code>opc-plc</code>	El certificado del servidor debe estar en <i>Trusted</i> en UaExpert; con <code>-autoaccept</code> el servidor acepta el del cliente, pero el cliente debe aceptar el del servidor.
Ignition no ve los contenedores	Usar las IPs de la red <code>ot-lab</code> (172.28.0.x), no <code>localhost</code> : Ignition corre dentro de Docker.
Ignition no ve los PLC físicos	En modo NAT (opción A) debe alcanzarlos por la IP real (192.168.10.x); comprobar con <code>docker run -rm -network ot-lab nicolaka/netshoot ping 192.168.10.11</code> .
Módulos de Ignition detenidos, HMI congelada	Ha expirado el trial de 2 h. Pulsar Reset Trial en la cabecera del Gateway.
Wireshark no decodifica OPC UA	Analyze → Decode As → TCP port 4840/50000 → OpcUa.
Wireshark no muestra la interfaz <code>br-xxxx</code>	Solo existe en Linux nativo. En macOS/Windows capturar con <code>netshoot</code> dentro de la red Docker.
<code>netshoot</code> no captura nada	Verificar que el nombre en <code>-net container:<nombre></code> coincide con <code>container_name</code> y que hay tráfico (un sondeo activo desde Ignition u OpenPLC).

Síntoma	Causa y solución
El Controllino responde a ping pero no a Modbus	El W5100 admite solo 4 sockets simultáneos; cerrar clientes (mbpoll, Ignition, FUXA cuentan). Ignition abre varias conexiones por dispositivo: limitar a 1 en la configuración del dispositivo.
El ESP32 se reinicia al recibir tráfico macvlan no funciona	Alimentación insuficiente por USB. Alimentarlo a 24 V. Solo en Linux nativo con NIC física cableada (no en Docker Desktop, no sobre WiFi, que suele filtrar MAC ajenas). Usar la opción A (NAT).
RS485 con datos corruptos	Revisar polaridad A/B, GND común y activar la terminación de $120\ \Omega$ en ambos extremos (el ESP32 PLC 14 y el Controllino tienen jumper o interruptor para ello).
Node-RED sin permisos sobre <code>nodered-data</code> (Linux)	El contenedor corre como uid 1000. <code>sudo chown -R 1000:1000 nodered-data</code> .
<code>docker compose pull</code> rompe algo que funcionaba	Una imagen <code>latest</code> cambió. Volver a la versión anterior fijando la etiqueta o restaurar <code>docker-compose.linux.yml.bak</code> .

Apéndice A

Archivos de configuración

Los archivos siguientes se incluyen desde la carpeta `src/` de esta guía y son copia exacta de los que hay en `ot-lab`.

A.1. `docker-compose.yml` (versión macOS / Docker Desktop)

```
# ot-lab -- versión para macOS (Docker Desktop, Intel o Apple Silicon)
#
# Cambios respecto a la versión Linux:
# - platform: linux/amd64 en openplc y conpot (solo publican imagen amd64;
#   en Apple Silicon corren bajo Rosetta/QEMU; activar "Use Rosetta" en
#   Docker Desktop > Settings > General para mejor rendimiento).
# - restart: unless-stopped para que el lab vuelva solo tras reiniciar el Mac.
# - Puerto 8883 de mosquitto eliminado (no hay listener TLS configurado).
# - Nota: en macOS el host NO alcanza las IPs 172.28.0.x directamente; usa
#   siempre localhost:<puerto publicado>. Entre contenedores sí funcionan.
#
# Copia original (Linux): docker-compose.linux.yml.bak

name: ot-lab

volumes:
  ignition-data:

networks:
  ot-lab:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.0.0/24

services:
  modbus-sim:
    image: oitc/modbus-server:latest
    container_name: modbus-sim
    restart: unless-stopped
    command: -f /server_config.json
    volumes:
      - ./modbus/server.json:/server_config.json:ro
    ports: ["5020:5020"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.30 }
```

```

openplc:
  image: tuttas/openplc_v3
  platform: linux/amd64          # imagen solo amd64
  container_name: openplc
  restart: unless-stopped
  ports: ["8080:8080", "502:502"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.10 }

opc-plc:
  image: mcr.microsoft.com/iotedge/opc-plc:latest
  container_name: opc-plc
  restart: unless-stopped
  command: >
    --pn=50000 --autoaccept --sph --sn=5 --sr=10 --st=uint
    --fn=5 --fr=1 --ft=uint --gn=5 --ut --dca
  ports: ["50000:50000"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.20 }

opcua-py:
  build: ./opcua
  container_name: opcua-py
  restart: unless-stopped
  ports: ["4840:4840"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.21 }

ignition:
  image: inductiveautomation/ignition:latest
  container_name: ignition
  restart: unless-stopped
  environment:
    ACCEPT_IGNORE_EULA: "Y"
    GATEWAY_ADMIN_PASSWORD: "lab1234!"
    IGNITION_EDITION: "standard"
  volumes:
    # Volumen con nombre, no bind mount: la imagen necesita sembrar su data/ inicial
    - ignition-data:/usr/local/bin/ignition/data
  ports: ["8088:8088"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.50 }

fuxa:
  image: frangoteam/fuxa:latest
  container_name: fuxa
  restart: unless-stopped
  volumes:
    - ./fuxa-data:/usr/src/app/FUXA/server/_appdata
  ports: ["1881:1881"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.51 }

node-red:
  image: nodered/node-red:latest
  container_name: node-red
  restart: unless-stopped
  volumes:
    - ./nodered-data:/data

```

```

ports: ["1880:1880"]
networks:
  ot-lab: { ipv4_address: 172.28.0.52 }

conpot:
  image: honeynet/conpot:latest
  platform: linux/amd64          # imagen solo amd64
  container_name: conpot
  restart: unless-stopped
  # La imagen no tiene ENTRYPOINT y el binario no está en PATH: usar ruta completa y
  # -f (foreground)
  command: ["/home/conpot/.local/bin/conpot", "--template", "default", "--logfile",
    "/var/log/conpot/conpot.log", "-f", "--temp_dir", "/tmp"]
  ports: ["10502:502", "10102:102", "10080:80", "10161:161/udp"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.90 }

mosquitto:
  image: eclipse-mosquitto:2
  container_name: mosquitto
  restart: unless-stopped
  volumes:
    - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
  ports: ["1883:1883"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.60 }

```

A.2. docker-compose.windows.yml (Windows / WSL 2)

Volúmenes con nombre para Ignition, FUXA y Node-RED; ver sección 4.3.3.

```

# ot-lab -- versión para Windows 10/11 (Docker Desktop con backend WSL 2)
#
# Uso (PowerShell o CMD, dentro de la carpeta ot-lab):
#   docker compose -f docker-compose.windows.yml up -d
#   docker compose -f docker-compose.windows.yml ps
#   docker compose -f docker-compose.windows.yml logs -f <servicio>
#
# Diferencias respecto a la versión macOS:
# - Volúmenes con nombre para Ignition, FUXA y Node-RED: evita problemas de permisos
#   (uid 1000) y de rendimiento de los bind mounts sobre NTFS (/mnt/c).
# - Solo se montan como bind mount los dos archivos de configuración (server.json,
#   mosquitto.conf). DEBEN tener fin de línea LF, no CRLF: Mosquitto falla con CRLF.
#   Si clonaste con Git en Windows: git config core.autocrlf false (o usa el .
#   gitattributes).
# - Puerto 50000 (OPC PLC): en Windows puede caer en el rango dinámico reservado por
#   Hyper-V/WSL. Si "docker compose up" falla con "bind: An attempt was made to access
#   a
#   socket in a way forbidden by its access permissions", comprueba con
#   netsh interface ipv4 show excludedportrange protocol=tcp
#   y cambia el mapeo a "4850:50000" (y usa opc.tcp://localhost:4850 en UaExpert).
# - platform: linux/amd64 se mantiene en openplc y conpot (inocuo en PC Intel/AMD;
#   necesario en Windows ARM).
# - Recomendado: clonar el proyecto DENTRO del sistema de archivos de WSL
#   (\\wsl$\\Ubuntu\\home\\<usuario>\\ot-lab) y no en C:\\Users\\..., por rendimiento.
# - Captura de tráfico: no existe interfaz br-xxxx visible; usar netshoot
#   (docker run --rm --net container:<svc> nicolaka/netshoot tshark -i eth0 ...).

```

```

# - El host alcanza los servicios solo por localhost:<puerto publicado>;
#   las IPs 172.28.0.x solo funcionan entre contenedores.

name: ot-lab

volumes:
  ignition-data:
  fuxa-data:
  nodered-data:

networks:
  ot-lab:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.0.0/24

services:
  modbus-sim:
    image: oitc/modbus-server:latest
    container_name: modbus-sim
    restart: unless-stopped
    command: -f /server_config.json
    volumes:
      - ./modbus/server.json:/server_config.json:ro
    ports: ["5020:5020"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.30 }

  openplc:
    image: tuttas/openplc_v3
    platform: linux/amd64
    container_name: openplc
    restart: unless-stopped
    ports: ["8080:8080", "502:502"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.10 }

  opc-plc:
    image: mcr.microsoft.com/iotedge/opc-plc:latest
    container_name: opc-plc
    restart: unless-stopped
    command: >
      --pn=50000 --autoaccept --sph --sn=5 --sr=10 --st=uint
      --fn=5 --fr=1 --ft=uint --gn=5 --ut --dca
    ports: ["50000:50000"] # si el puerto está reservado en Windows: "4850:50000"
    networks:
      ot-lab: { ipv4_address: 172.28.0.20 }

  opcua-py:
    build: ./opcua
    container_name: opcua-py
    restart: unless-stopped
    ports: ["4840:4840"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.21 }

  ignition:
    image: inductiveautomation/ignition:latest

```

```
container_name: ignition
restart: unless-stopped
environment:
  ACCEPT_IGNITION_EULA: "Y"
  GATEWAY_ADMIN_PASSWORD: "lab1234!"
  IGNITION_EDITION: "standard"
volumes:
  - ignition-data:/usr/local/bin/ignition/data
ports: ["8088:8088"]
networks:
  ot-lab: { ipv4_address: 172.28.0.50 }

fuxa:
  image: frangoteam/fuxa:latest
  container_name: fuxa
  restart: unless-stopped
  volumes:
    - fuxa-data:/usr/src/app/FUXA/server/_appdata
  ports: ["1881:1881"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.51 }

node-red:
  image: nodered/node-red:latest
  container_name: node-red
  restart: unless-stopped
  volumes:
    - nodered-data:/data
  ports: ["1880:1880"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.52 }

conpot:
  image: honeynet/conpot:latest
  platform: linux/amd64
  container_name: conpot
  restart: unless-stopped
  # La imagen no tiene ENTRYPOINT y el binario no está en PATH: ruta completa y -f (
  foreground)
  command: ["/home/conpot/.local/bin/conpot", "--template", "default", "--logfile",
    "/var/log/conpot/conpot.log", "-f", "--temp_dir", "/tmp"]
  ports: ["10502:502", "10102:102", "10080:80", "10161:161/udp"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.90 }

mosquitto:
  image: eclipse-mosquitto:2
  container_name: mosquitto
  restart: unless-stopped
  volumes:
    - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
  ports: ["1883:1883"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.60 }
```

A.3. docker-compose.yml (versión original Linux)

Diferencias respecto a la versión macOS: sin `platform`, sin `restart`, Ignition con `bind mount` y Conpot con `command` corto; en Linux nativo estas dos últimas también fallan (sección 4.7), por lo que se recomienda usar la versión macOS en todos los sistemas.

```
name: ot-lab

networks:
  ot-lab:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.0.0/24

services:
  modbus-sim:
    image: oitc/modbus-server:latest
    container_name: modbus-sim
    command: -f /server_config.json
    volumes:
      - ./modbus/server.json:/server_config.json:ro
    ports: ["5020:5020"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.30 }

  openplc:
    image: tuttas/openplc_v3
    container_name: openplc
    ports: ["8080:8080", "502:502"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.10 }

  opc-plc:
    image: mcr.microsoft.com/iotedge/opc-plc:latest
    container_name: opc-plc
    command: >
      --pn=50000 --autoaccept --sph --sn=5 --sr=10 --st=uint
      --fn=5 --fr=1 --ft=uint --gn=5 --ut --dca
    ports: ["50000:50000"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.20 }

  opcua-py:
    build: ./opcua
    container_name: opcua-py
    ports: ["4840:4840"]
    networks:
      ot-lab: { ipv4_address: 172.28.0.21 }

  ignition:
    image: inductiveautomation/ignition:latest
    container_name: ignition
    environment:
      ACCEPT_IGNITION_EULA: "Y"
      GATEWAY_ADMIN_PASSWORD: "lab1234!"
      IGNITION_EDITION: "standard"
    volumes:
      - ./ignition-data:/usr/local/bin/ignition/data
```

```

ports: ["8088:8088"]
networks:
  ot-lab: { ipv4_address: 172.28.0.50 }

fuxa:
  image: frangoteam/fuxa:latest
  container_name: fuxa
  volumes:
    - ./fuxa-data:/usr/src/app/FUXA/server/_appdata
  ports: ["1881:1881"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.51 }

node-red:
  image: nodered/node-red:latest
  container_name: node-red
  volumes:
    - ./nodered-data:/data
  ports: ["1880:1880"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.52 }

conpot:
  image: honeynet/conpot:latest
  container_name: conpot
  command: conpot --template default --logfile /var/log/conpot/conpot.log
  ports: ["10502:502", "10102:102", "10080:80", "10161:161/udp"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.90 }

mosquitto:
  image: eclipse-mosquitto:2
  container_name: mosquitto
  volumes:
    - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
  ports: ["1883:1883", "8883:8883"]
  networks:
    ot-lab: { ipv4_address: 172.28.0.60 }

```

A.4. modbus/server.json

```

{
  "server": {
    "listenerAddress": "0.0.0.0",
    "listenerPort": 5020,
    "protocol": "TCP",
    "tlsParams": { "privateKey": null, "certificate": null },
    "logging": {
      "format": "%(asctime)-15s %(levelname)-8s %(message)s",
      "logLevel": "INFO"
    }
  },
  "persistence": { "enabled": false, "file": "/data/modbus_registers.json", "
    saveInterval": 30 },
  "metrics": { "enabled": false, "address": "0.0.0.0", "port": 9090, "path": "/metrics"
  },

```

```

"registers": {
  "description": "Claves = offset de protocolo (0-based). HR 0..2 -> [1234, 25, 72];
  IR 0 -> 999",
  "initializeUndefinedRegisters": true,
  "discreteInput": { "0": true, "1": false },
  "coils": { "0": false, "1": true },
  "holdingRegister": { "0": 1234, "1": 25, "2": 72 },
  "inputRegister": { "0": 999 }
}
}

```

A.5. opcua/server.py y opcua/Dockerfile

```

import asyncio, math
from asyncua import Server

async def main():
    srv = Server()
    await srv.init()
    srv.set_endpoint("opc.tcp://0.0.0.0:4840/lab/")
    srv.set_server_name("Lab OPC UA Python")
    ns = await srv.register_namespace("http://lab.local")
    obj = await srv.nodes.objects.add_object(ns, "Tanque1")
    nivel = await obj.add_variable(ns, "Nivel", 0.0)
    temp = await obj.add_variable(ns, "Temperatura", 20.0)
    valv = await obj.add_variable(ns, "Valvula", False)
    await valv.set_writable()
    t = 0
    async with srv:
        while True:
            t += 1
            await nivel.write_value(50 + 30 * math.sin(t / 10))
            await temp.write_value(20 + 5 * math.cos(t / 15))
            await asyncio.sleep(1)

asyncio.run(main())

```

```

FROM python:3.12-slim
RUN pip install --no-cache-dir asyncua
COPY server.py /app/server.py
CMD ["python", "/app/server.py"]

```

A.6. mosquitto/mosquitto.conf

```

listener 1883
allow_anonymous true

```

Para el ejercicio TLS, añadir un segundo listener con certificados generados con OpenSSL (o mkcert) y publicar el 8883 en el compose:

```

listener 1883
allow_anonymous true

listener 8883

```

```
cafile    /mosquitto/certs/ca.crt  
certfile  /mosquitto/certs/server.crt  
keyfile   /mosquitto/certs/server.key  
allow_anonymous true
```

Apéndice B

Referencia rápida de comandos

```
# --- Ciclo de vida ---
docker compose up -d                                # arrancar todo
docker compose up -d <svc>                           # arrancar uno
docker compose up -d --force-recreate <svc>          # recrear tras cambiar el compose
docker compose up -d --build opcua-py               # reconstruir imagen propia
docker compose ps / stop / start / restart <svc> / down / down -v
docker compose logs -f --tail 50 <svc>
docker stats

# --- Inspeccion ---
docker inspect <imagen> --format '{{.Config.Entrypoint}} {{.Config.Cmd}}'
docker network inspect ot-lab
docker volume ls && docker volume inspect ot-lab_ignition-data
docker exec -it <svc> sh

# --- Captura ---
docker run --rm --net container:<svc> nicolaka/netshoot tshark -i eth0 -Y mbtcp
docker run --rm --net container:<svc> -v "$PWD/captures:/cap" nicolaka/netshoot \
  tcpdump -i eth0 -w /cap/<nombre>.pcap "<filtro bpf>"

# --- Escaneo ---
docker run --rm --network ot-lab nicolaka/netshoot nmap -sT -p 502,5020,102,4840,50000
172.28.0.0/24
docker run --rm --network ot-lab nicolaka/netshoot nmap -p 502 --script modbus-discover
<ip>
docker run --rm --network ot-lab nicolaka/netshoot nmap -p 102 --script s7-info
172.28.0.90

# --- Clientes rapidos ---
docker run --rm --network ot-lab python:3.12-slim sh -c "pip -q install pymodbus &&
python -c \"...\\""
docker run --rm --network ot-lab python:3.12-slim sh -c "pip -q install asyncua &&
python -c \"...\\""
docker run --rm --network ot-lab eclipse-mosquitto:2 mosquitto_sub -h 172.28.0.60 -t '
ot/#' -v
docker run --rm --network ot-lab eclipse-mosquitto:2 mosquitto_pub -h 172.28.0.60 -t ot
/test -m hola

# --- Copia de seguridad de Ignition ---
docker run --rm -v ot-lab_ignition-data:/d -v "$PWD:/b" alpine tar czf /b/ignition-
backup.tgz -C /d .
```